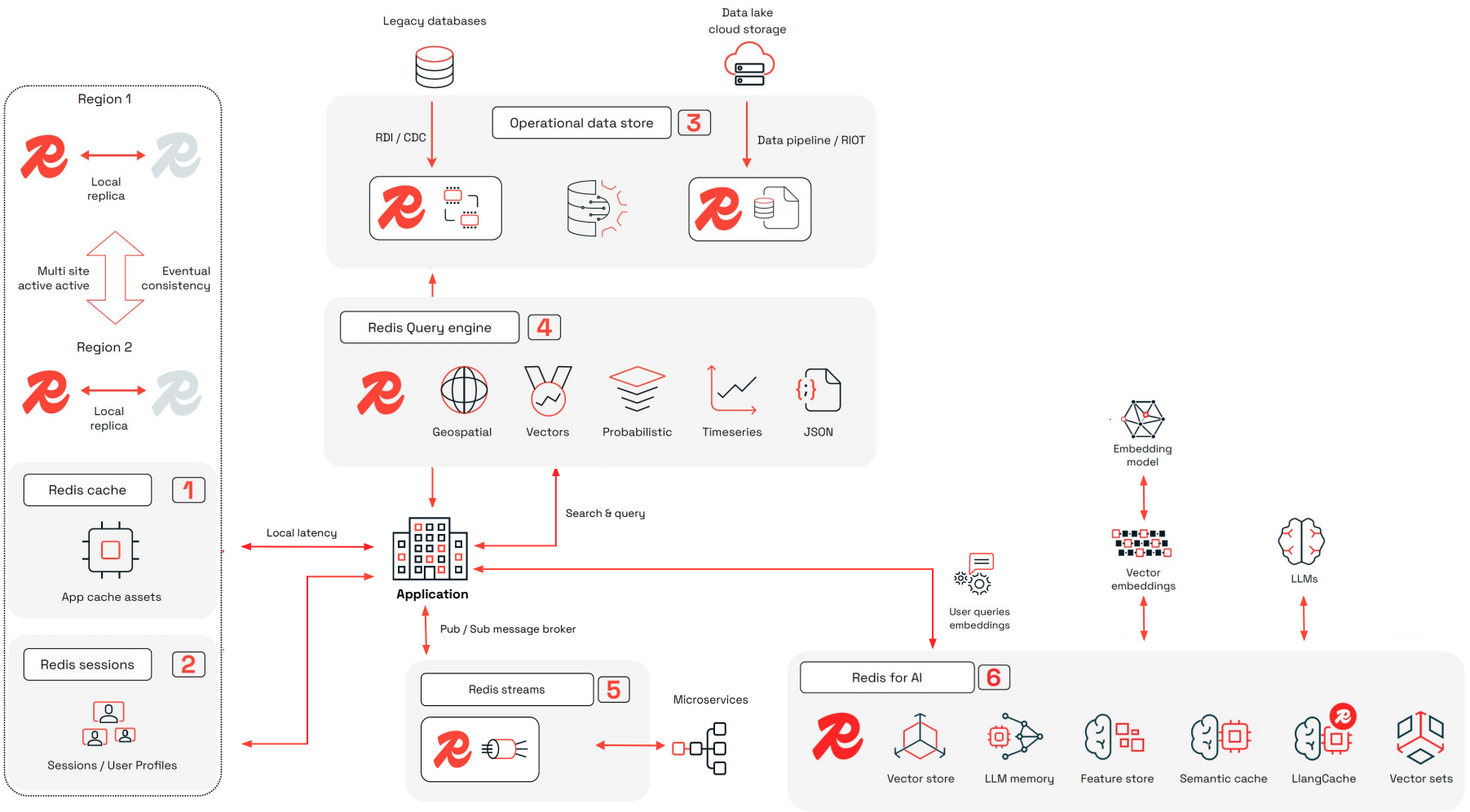
Redis Enterprise

Overview



Redis Enterprise 8 & Redis 8 are here!

Redis Enterprise 8 & Redis 8: Powering the Next Generation of Real-time Applications

Redis Enterprise 8 & Redis 8
The Fastest, Most AI-Ready Redis yet!

Build for AI

- Larger datasets with **Redis Flex (v2)**
- **LangCache:** next level caching for AI
- New **Vector Sets** data structure

Fastest Redis Ever

- 40% faster scaling
- Up to 78% Latency Improvements
- More robust replication
- RQE performance improvements
- Query Performance Factor: **up to 16x Perf** ⬆️

Security & Control

- Connect with AWS Private Link
- Customer Managed Keys in Redis Cloud
- Manage your own CA

Reliability & Simplicity

For Redis Cloud

- **Seamless client experience** on database changes
- Redis manages minor versions upgrades
- Customer upgrades major self-serve

Focus on Building Apps

- Numerous API improvements that save you time
- Model Context Protocol (MCP) for Redis

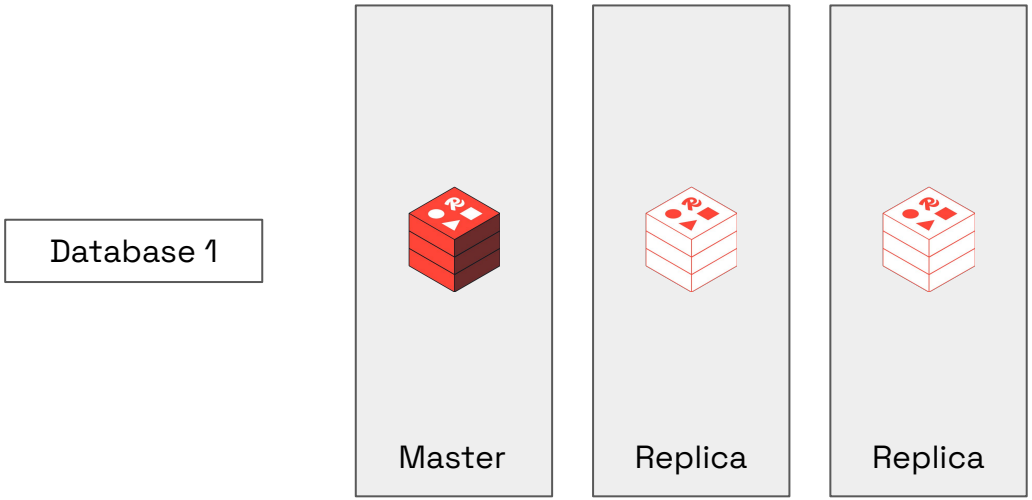
Cloud | On Premise

Redis API | Redis Open Source Projects

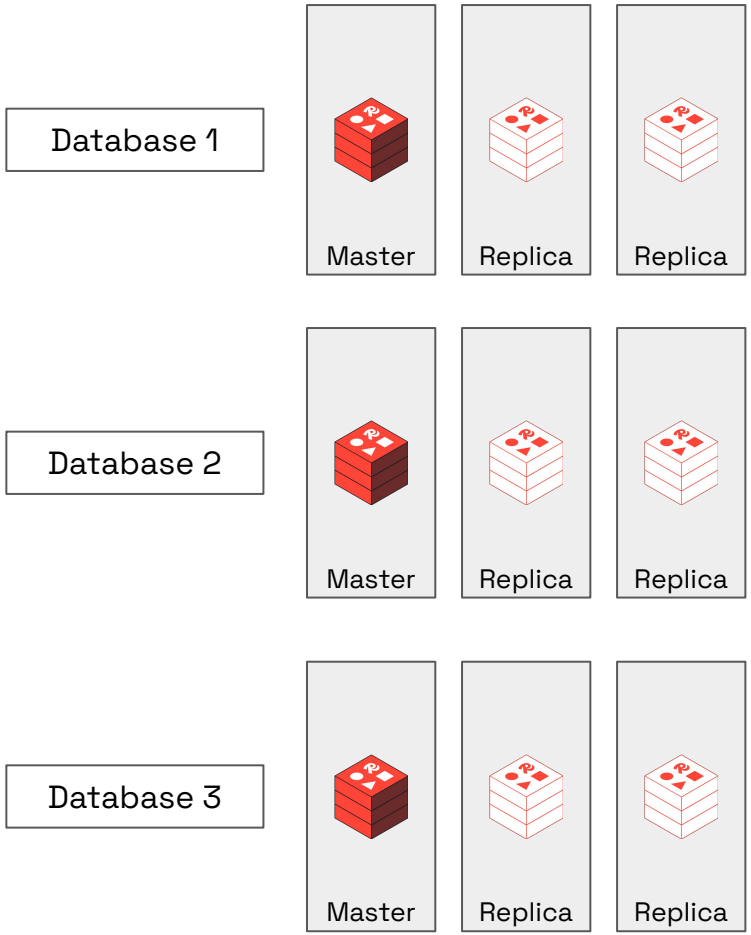
Infrastructure Optimisation

Optimize Infrastructure

How is your redis deployed today ?



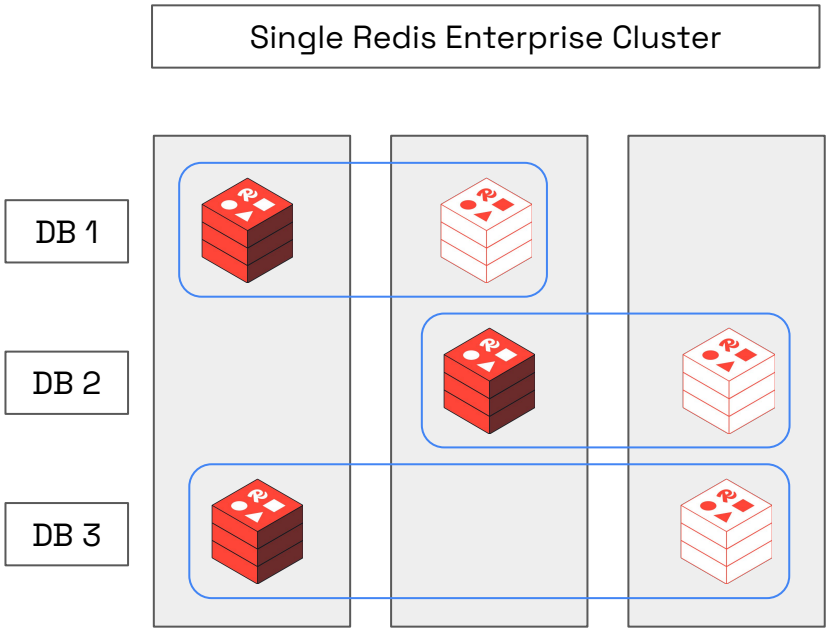
Redis Community Edition



What may be the challenges ?

- Every machine no matter how large, will utilize only 1 core*
- For every new database/microservice, you need a new cluster
- For every cluster, you need at least 2 replicas per master to avoid split brain situation and get true HA

How do we solve them?



- Every machine hosts multiple shards belonging to same or different dbs, thus utilizing more cores
- For every new database/microservice, you do not need to provision new infrastructure
- Redis Enterprise Quorum is based on the number of machines in the cluster, hence the number of replicas required per master is just 1

High Availability

The High Availability - Pyramid

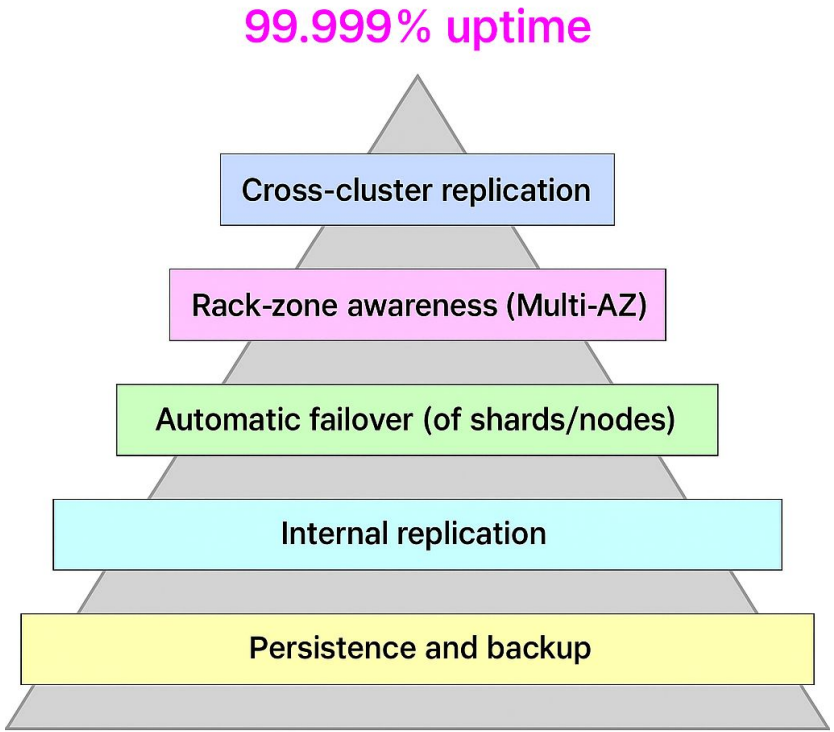
Allow switching to another cluster in another region in case of a disaster (outage of an entire cluster)

Minimal downtime in case of a failure of up to one zone the same time

Minimal downtime in case of a failure of up to one node the same time

Up to two copies of the datak, whereby each copy is on a different node

Allows recovering from persistent data files



Quorum Concept

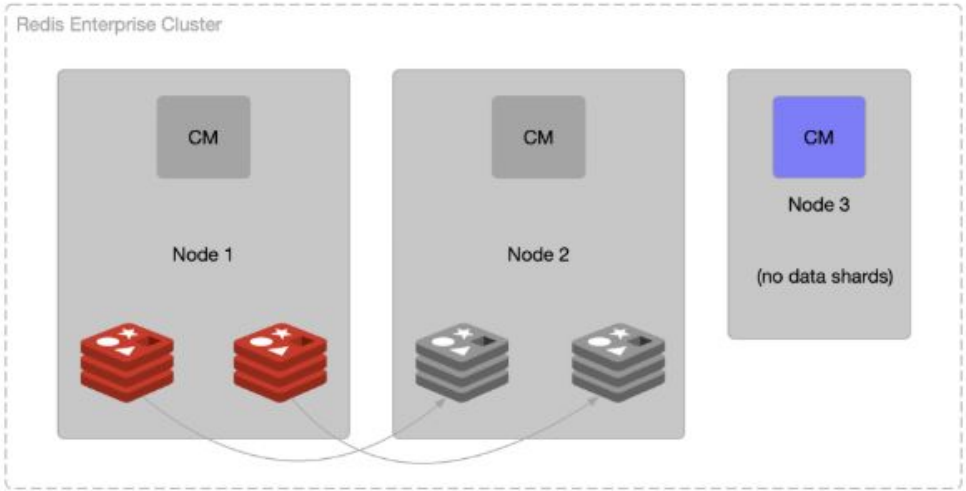
Redis Enterprise uses a node-based quorum concept, which means that we need to have an odd number of nodes to protect the cluster from split-brain situation

The Problem

- It's impossible within the distributed system to device why a node is no longer reachable
  Node failure or network partitioning event?
- In order to avoid split-brain situations, Redis Enterprise's Master of the Cluster (the brain) is elected within the majority network partition (the one which has the majority of the nodes).
  Nodes within the minority partition stop working and will no longer server requests

Quorum Concept - QO Node

- Q(uorum) - O(nly) node
- Only necessary if we have an even number of data nodes
- Ensures an odd number of nodes
- Lower HW spec, so for cost-saving purposes
- Doesn't host any data

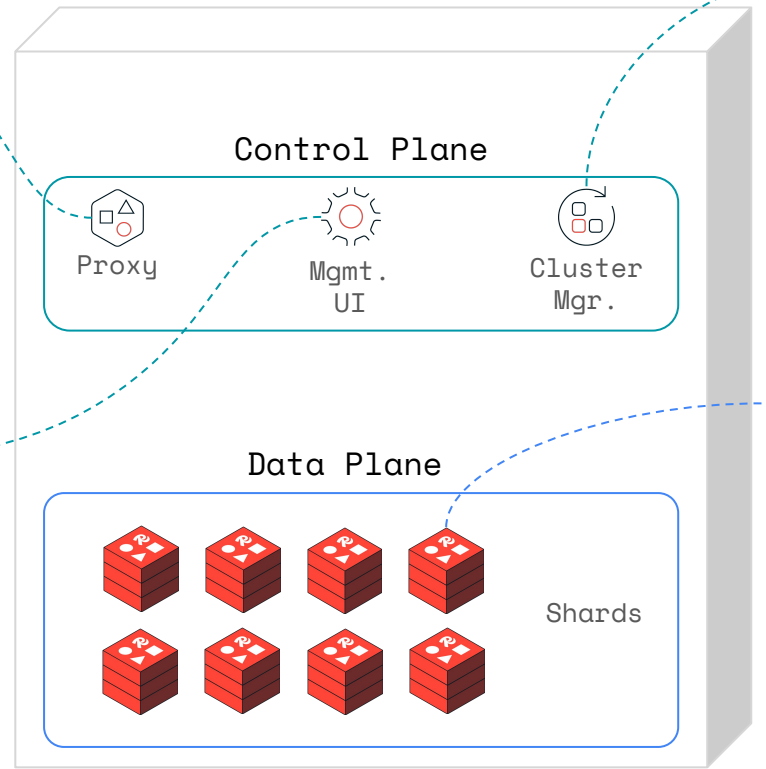


Redis Architecture – A Single Node (Machine)

Separation of Concerns

Allows multiplexing of commands & pipelining. Improved performance 2X

Simplified manageability, operate through management UI & REST API support



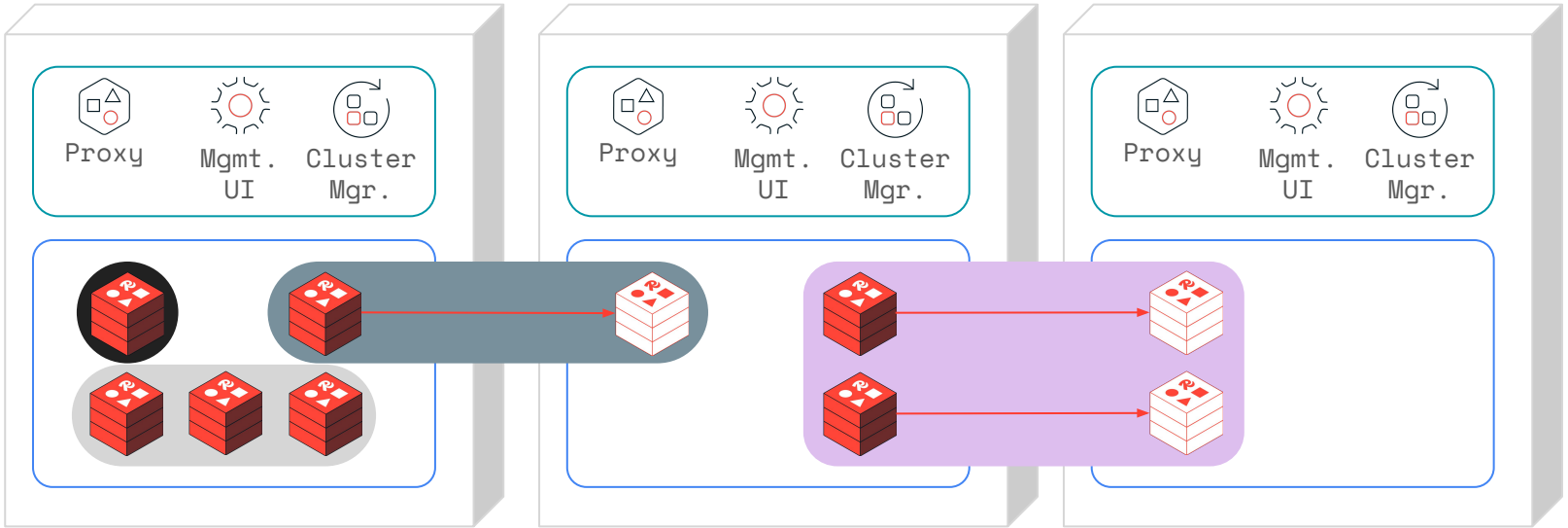
Automated Cluster health, node health and database health monitoring, failovers, stats collection and more

A single redis process, a unit of redis memory

- Every cluster has >= 3 machines (nodes)
- Every cluster can have > 1 db. Every db. can have > 1 shard
- Every shard = 1 CPU

Redis Architecture – A Cluster

Separation of Concerns



Building Resilient Global Apps

Building highly available and resilient global apps.

Active-Passive

Improves application **response time**

Enables highly available standby nodes in **disaster recovery** scenarios

Provides flexible cluster configuration to **manage cost**

Active-Active

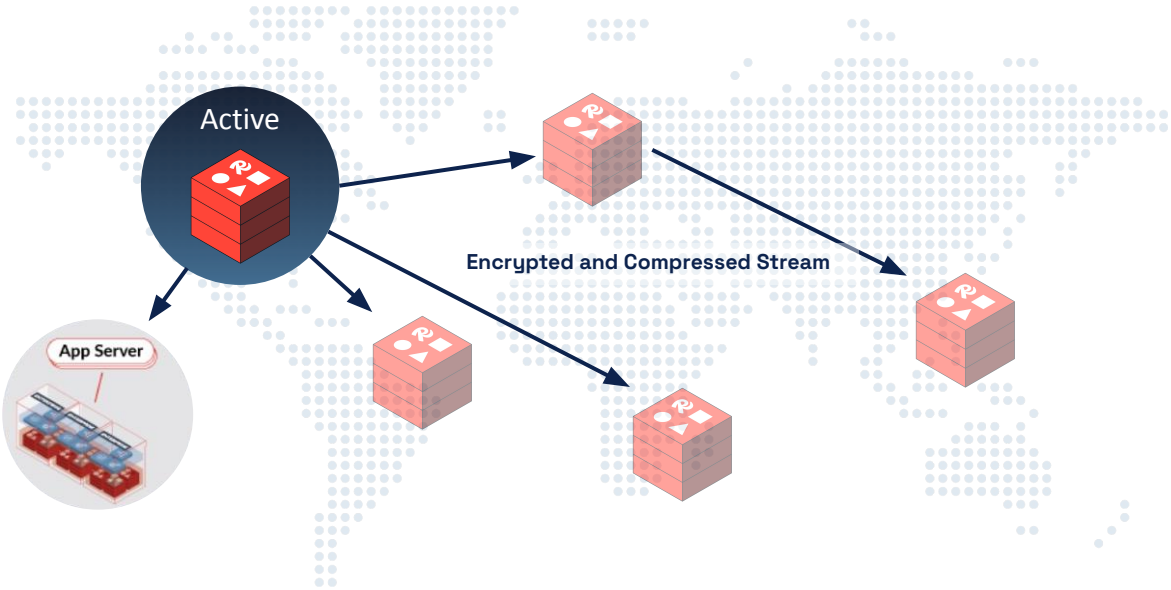
Delivers **local** latency across geos

Provides **instant** failover without data loss

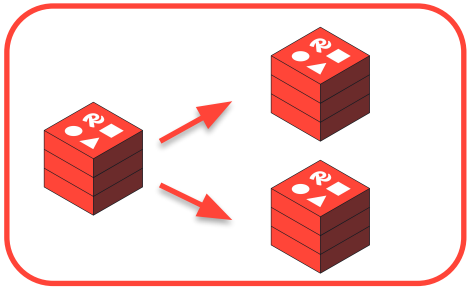
Unifies your data layer across environments through **seamless** conflict resolution

Understanding Active-Passive (Replica-of).

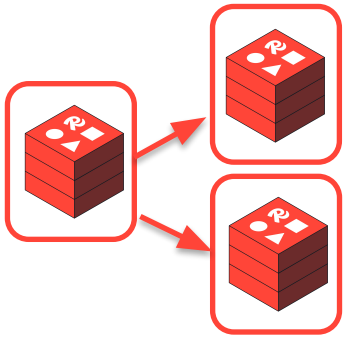
- Active instance accepts read and write
- Updates are compressed and encrypted
- Passive instance can be read from



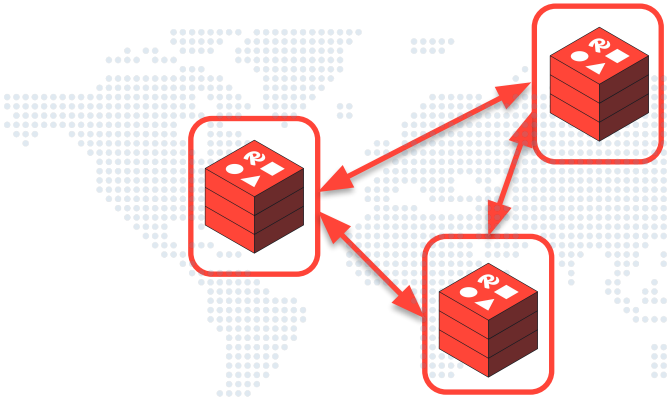
Redis Cloud SLA.



99.9% (<10m4.8s)
Active-passive replication Single-AZ



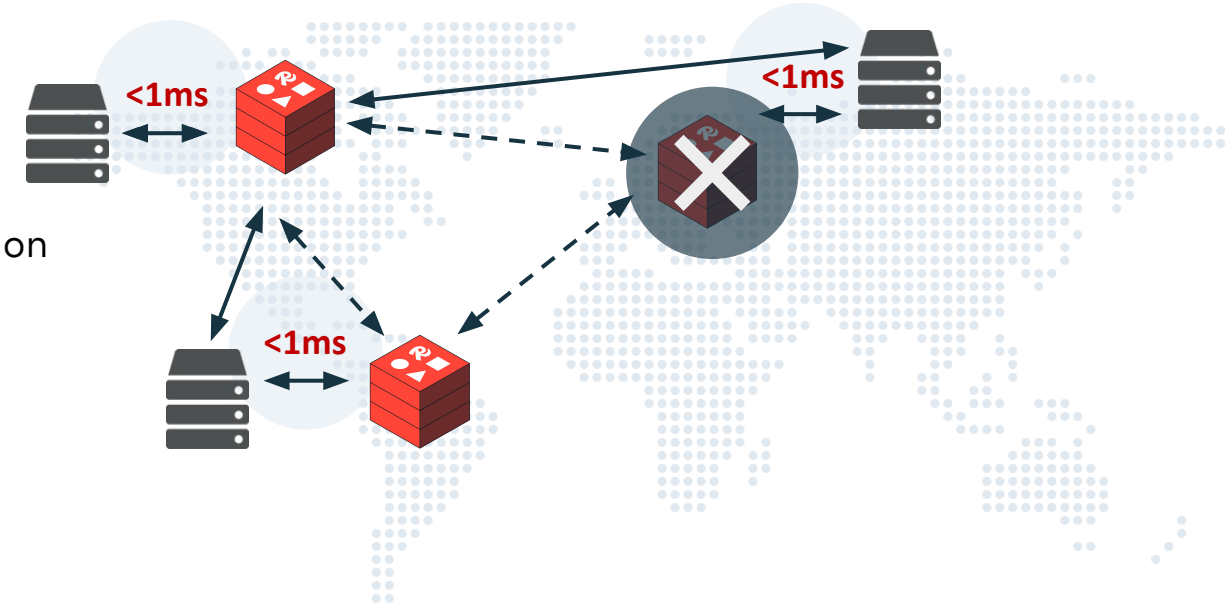
99.99% (<4m23s)
Active-passive replication Multi-AZ



99.999% (<26.3s)
Active-Active replication Multi-region

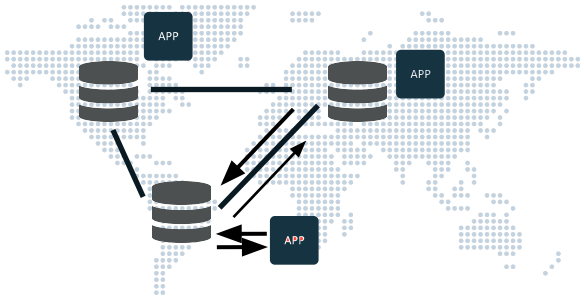
CRDTs based Active-Active delivers key differentiation.

- Local <1ms latency
- CRDTs-based conflict resolution
- Strong resiliency based on consensus-free protocol



Sub-millisecond performance delivers unique value.

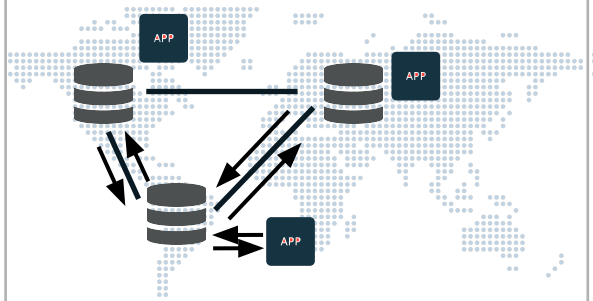
NoSQL DB



Eventual Consistency

100ms

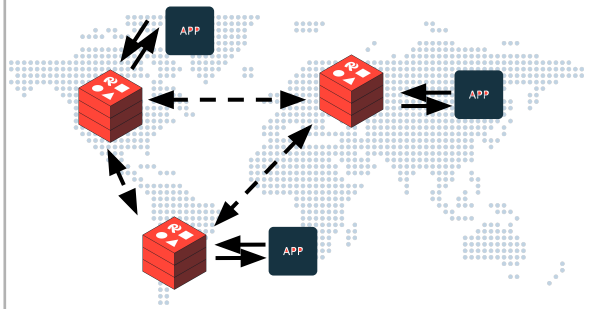
NoSQL DB



Strong Consistency

200ms

Redis CRDTs

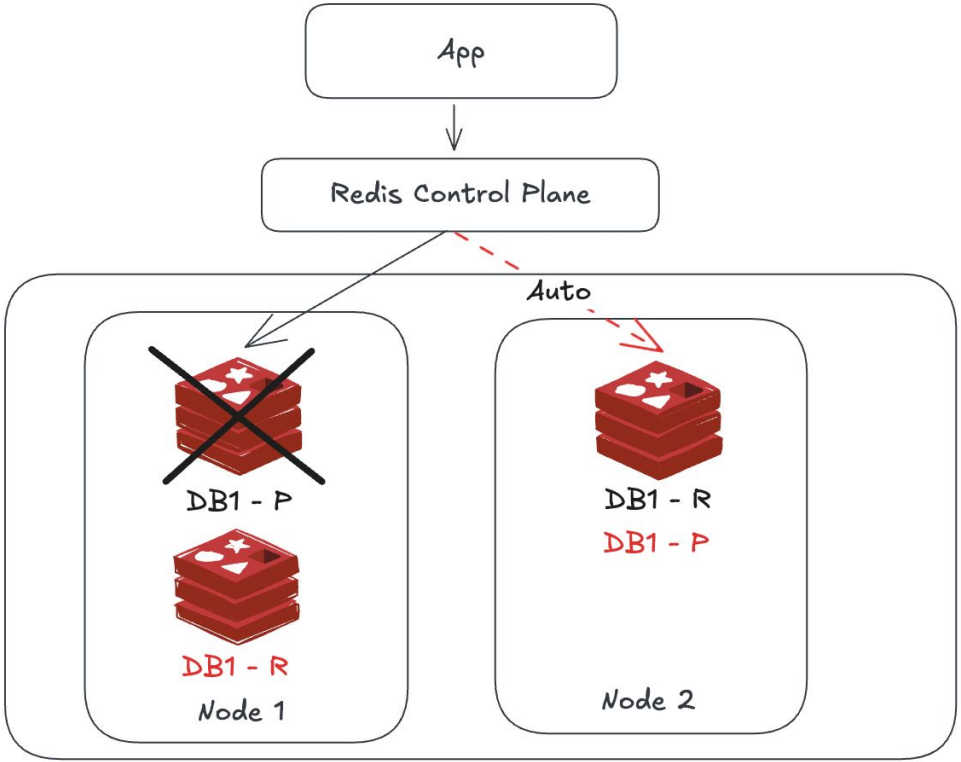


Strong Eventual Consistency, Causal Consistency

<1ms

Disaster Recovery Options

Option 1: High Availability



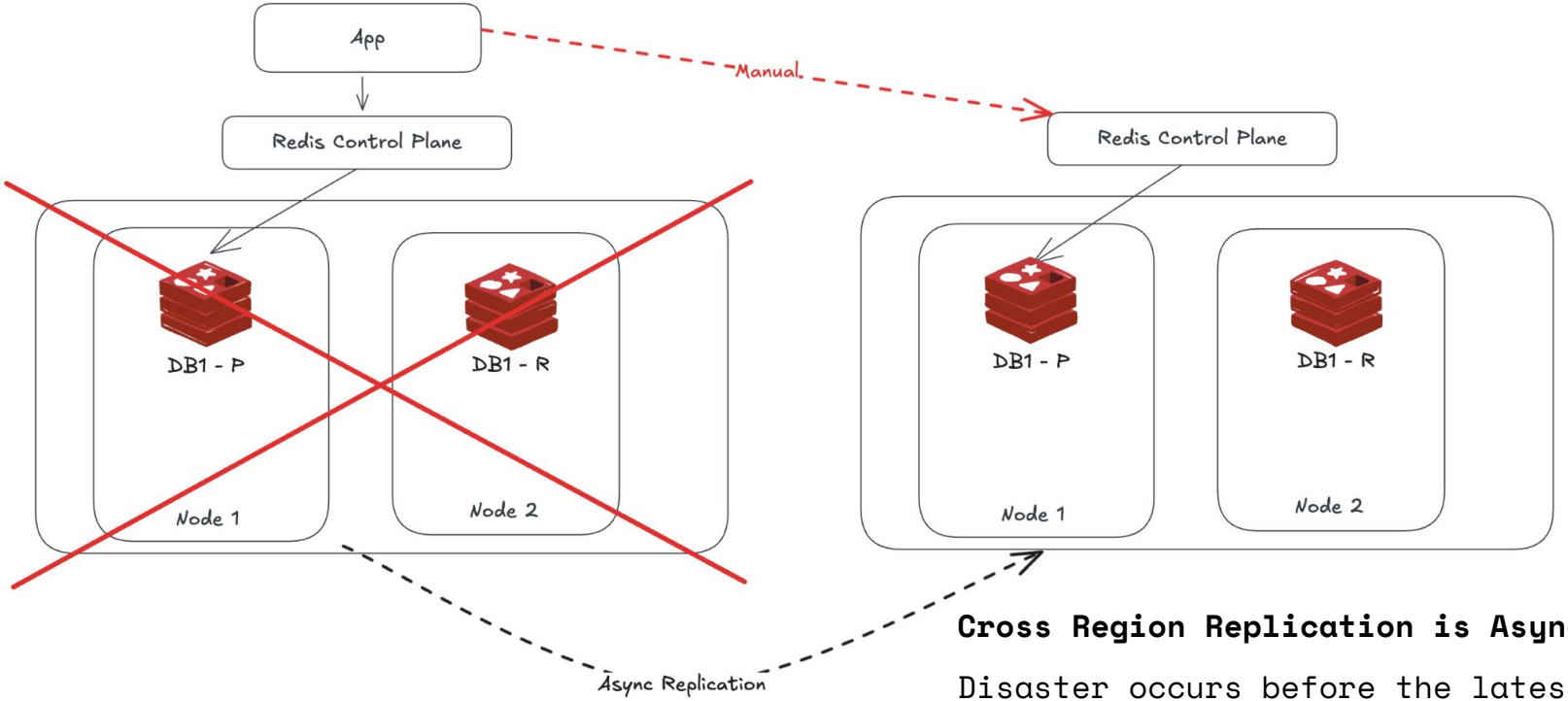
Zero Data Loss Architecture

Uses real-time sync between primary and replica nodes, ensuring that data is replicated before write operations are acknowledges

Automatic Failover and Proxy Redirection

Intelligent proxy layer monitor database health - when a primary node fail the system redirect to the replica node

Option 2: HA + Active/Passive



Cross Region Replication is Asynchronous

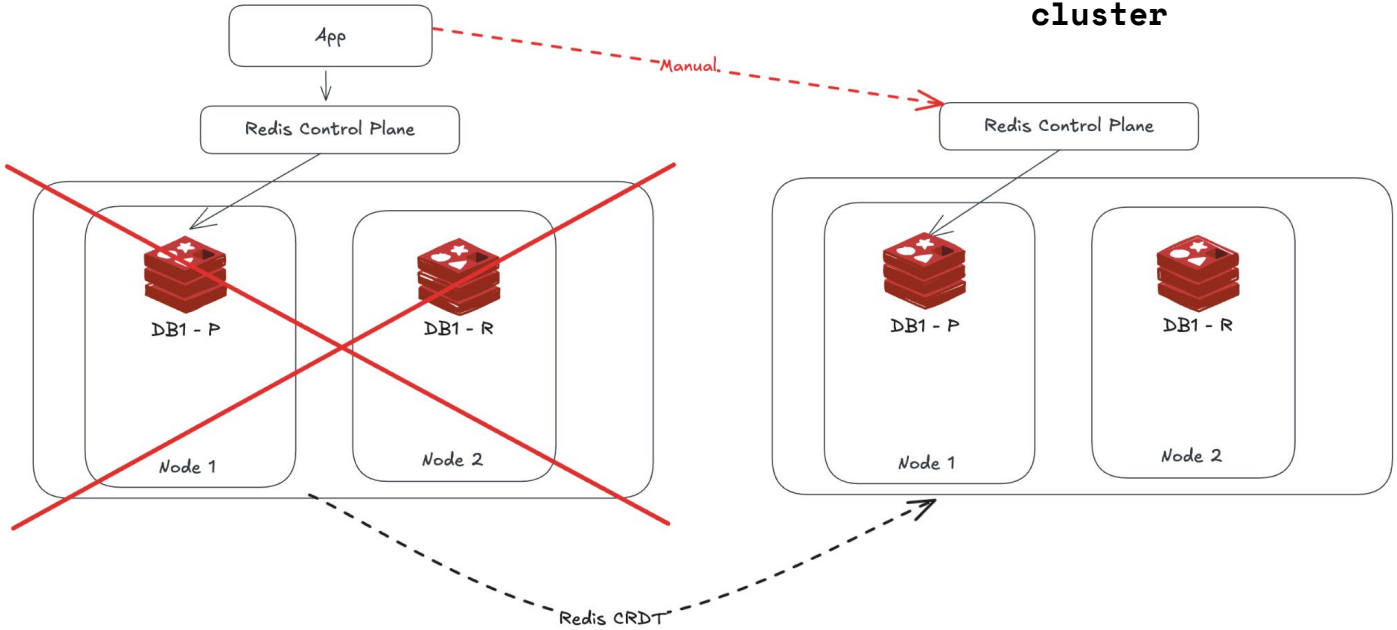
Disaster occurs before the latest write operations are replicated to the standby region, a few seconds of data may be lost

Option 2: HA + Active/Passive

- This standby region is not a simple backup, it operates as a live, asynchronous replica of the primary Redis cluster
- Each primary shard has a **one-to-one replica** shard in the passive region continuously updated via async replication
- The passive cluster remains hot and failover ready with minimal replication lag
- This architecture requires ~2x shard capacity
  - Full shard set in the primary region
  - A full replica shard set in the standby region

Option 3: HA + Active/Active

Each cluster can perform **read and write operation**, these changes are synchronized **bidirectional across cluster**



- **Achieve this through CRDT (Conflict Free Replicated Data Type)**
- **Data model that allows updates from multiple region to be automatically merged**

Option 3: How CRDTs Work (Active/Active)

- Each region operates independently
  - No global lock
  - No cross region coordination
  - Low latency local writes
- All updates replicate asynchronous
  - Uses logical timestamps + metadata
  - Replication is eventually consistent
  - No write conflicts block the application
- Automatic state convergence
  - Compare CRDT metadata
  - Merge conflicting operation deterministically
  - Reach the same final state in all regions
- Region recovery is automatic
  - If a region disconnect, continue accepting local write, when reconnect, CRDT algo merge all divergent changes

Scenarios between Region A and Region B
App writes to region A and region A goes down

Write are already committed to region A locally before it failed

All committed write are stored in the CRDT dataset in Region A

Case 1: Replication already happened

- Region A → B replication usually happens within milliseconds
- If they already received the update, nothing is lost

Case 2: Replication had not happened

- Region A still has the committed CRDT write on disk
- When region A recovers:
  - Reconnect
  - Send its missed updates to B and C
  - Receives updates from B and C
  - Merges everything via CRDTs

Redis Deployment Models Summary Table

Feature / Behavior	**HA (Single Region)**	**Active/Passive (Multi Region DR)**	**Active/Active (Multi-Region CRDT)**
Regions	1 Region	2+ Regions (Primary + Replica)	2+ regions (all active)
Replication Type	Sync within region	Async cross-region replication	CRDT-based async merging
Failover Behaviour	Local failover only	Standby promoted to primary	No promotion needed; other region continue immediately
RPO (Data loss risk)	-	> 0 (some data may be lost)	0 - no data loss for CRDT type
RTO (Recovery Time)	-	Minutes (Promotion + DNS app/routing)	Full global durability with automatic conflict resolution) - Near Zero

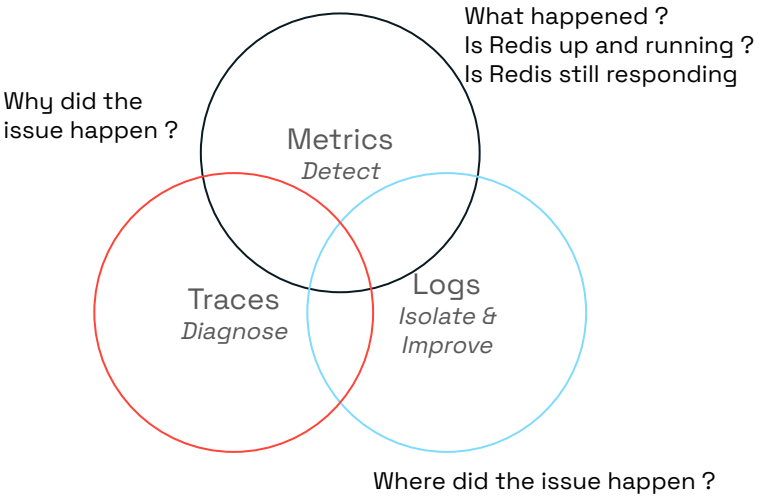
Redis Deployment Models Summary Table

Feature / Behavior	HA (Single Region)	Active/Passive (Multi Region DR)	Active/Active (Multi-Region CRDT)
If a region fails	Cluster self-heals inside region	Passive becomes writable after failover	Other region continue writing immediately
When Failed Region Returns	Syncs from surviving replica	Must rebuild from new primary	Automatic merges via CRDTs
Traffic Routing	Local only	Needs failover routing (DNS/LB)	Geo-routing or multi-endpoint clients
Best For	Single Region HA	DR with limited RPO/RTO requirements	Mission critical multi region apps demanding RPO/RTO = 0

Observability and Management

Out of the box real time monitoring

Superior metrics beyond Redis INFO command

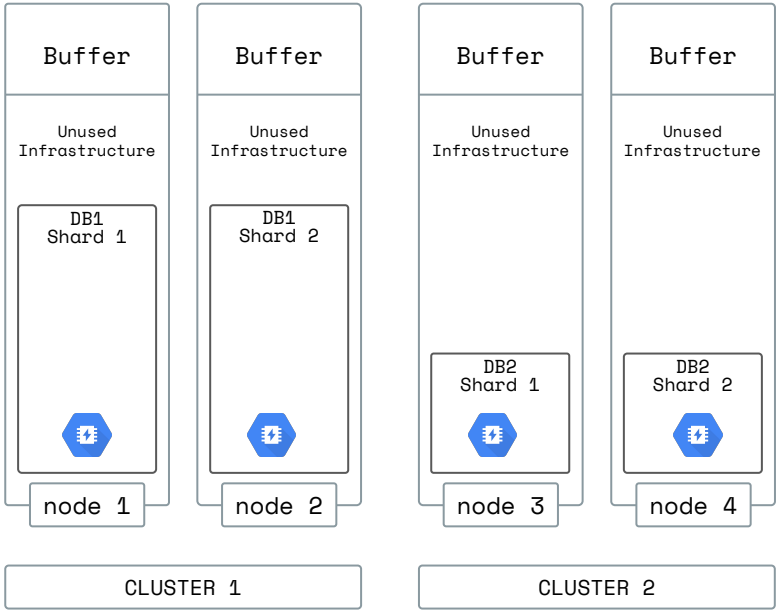


Smart on costs

Put the infrastructure you pay for to use.

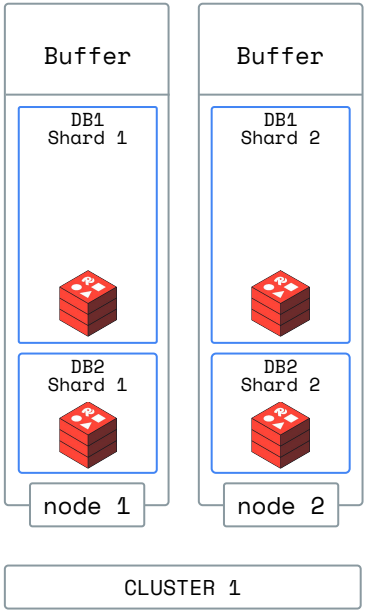
Single-tenant MemoryStore

(4 instances to host 2 databases)



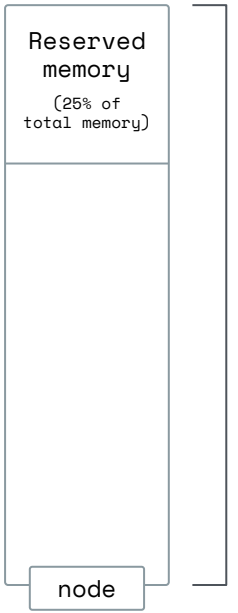
Multi-tenant Redis

(2 GCP instances to host the same 2 databases)



Only pay for what you use.

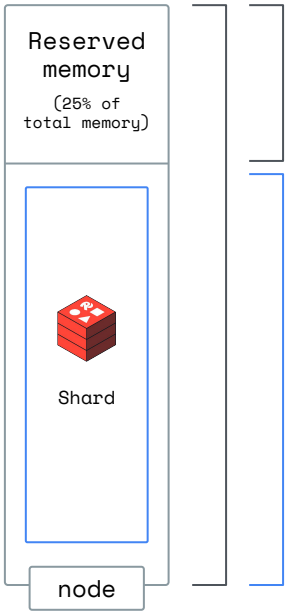
MemoryStore



- 25% of the node's memory is not usable by Redis processes
- It's set aside for backups and failover operations

A **cache node** is what you pay for with MemoryStore

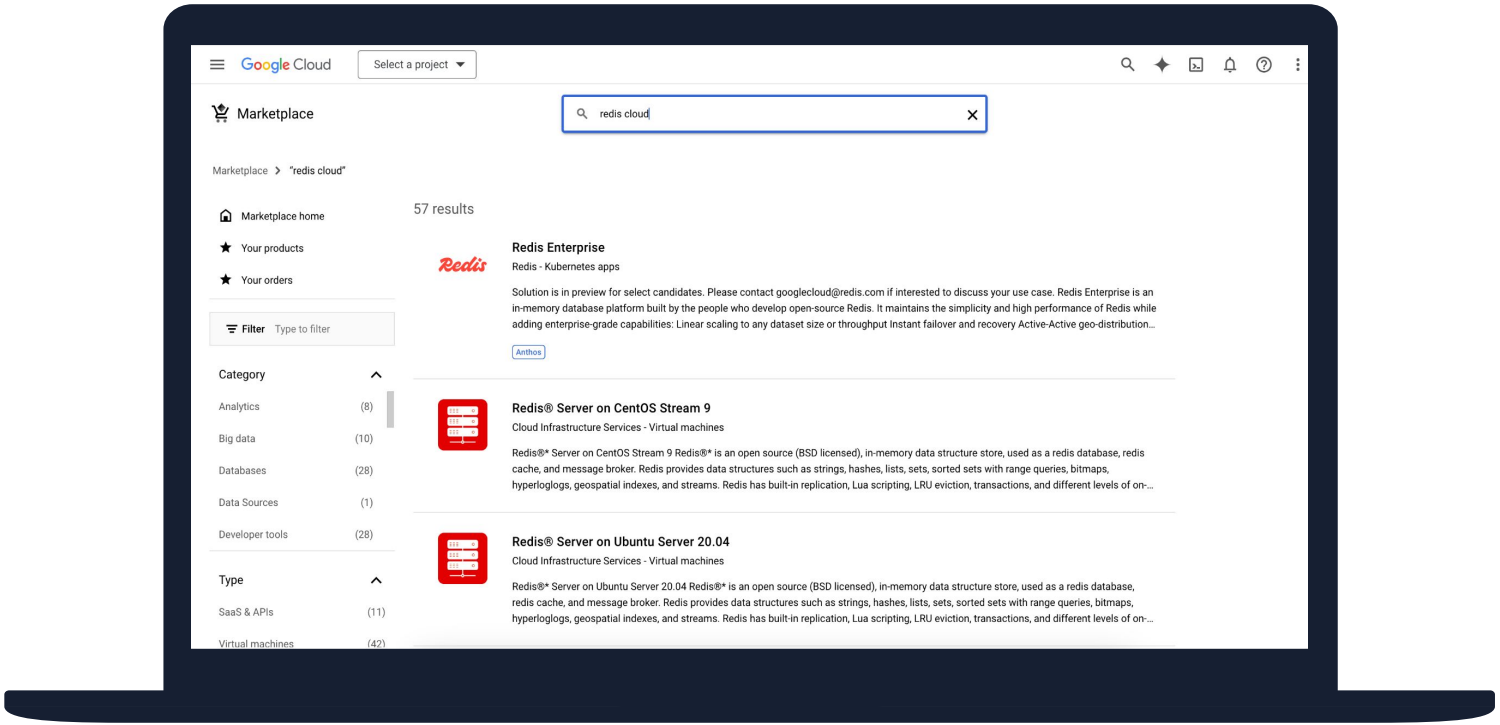
Redis



- The reserved memory still exists, but it's hidden
- Pricing is based on the memory you use in Redis Cloud, not the instance memory

- A **shard** is what you pay for in Redis Cloud
- You can use 100% of the memory of every shard you buy

Simplify billing through Cloud Provider Marketplace

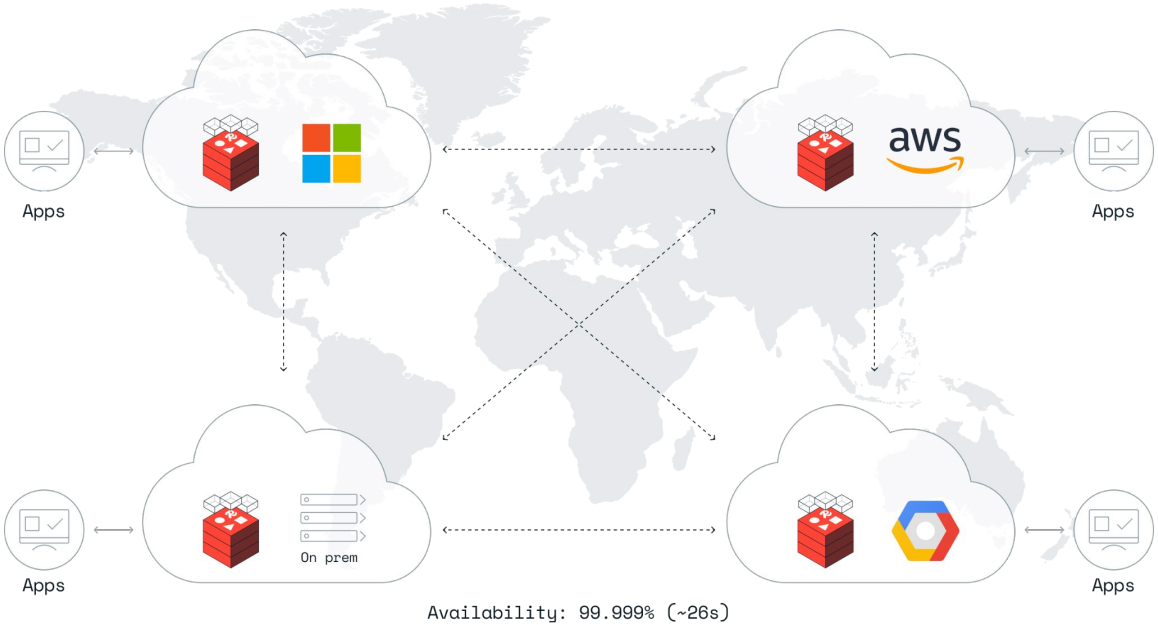


Burn down committed spend on GCP while receiving a much better version of Redis

Multi-cloud and hybrid support

Flexible deployment

- Deploy in any cloud
- Deploy on-premises or hybrid
- Native Kubernetes support with operator



GCC Public Sector Deployment Requirements

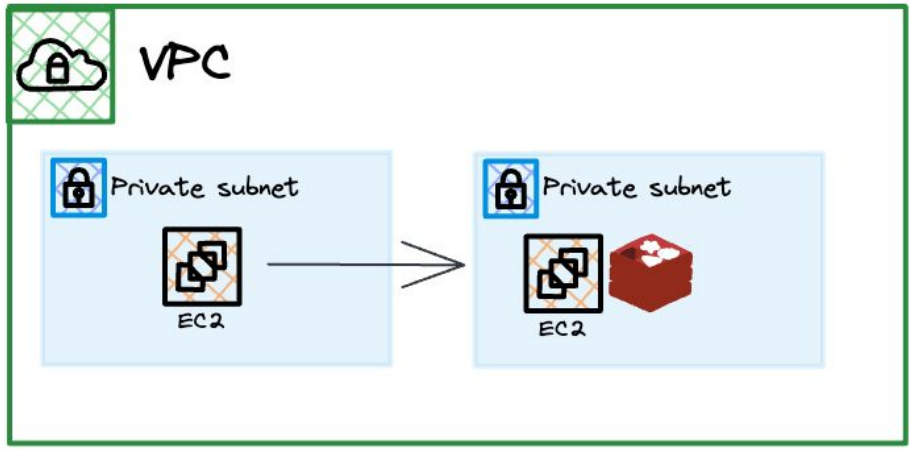
Common GCC Requirements

1. Security, Isolation and Compliance Standards
2. Data Residency Requirements
3. Network Boundary Controls
4. High Availability
5. Disaster Recovery

Deployment Option: Redis Software

Redis Enterprise software installed and managed by the agency/partner

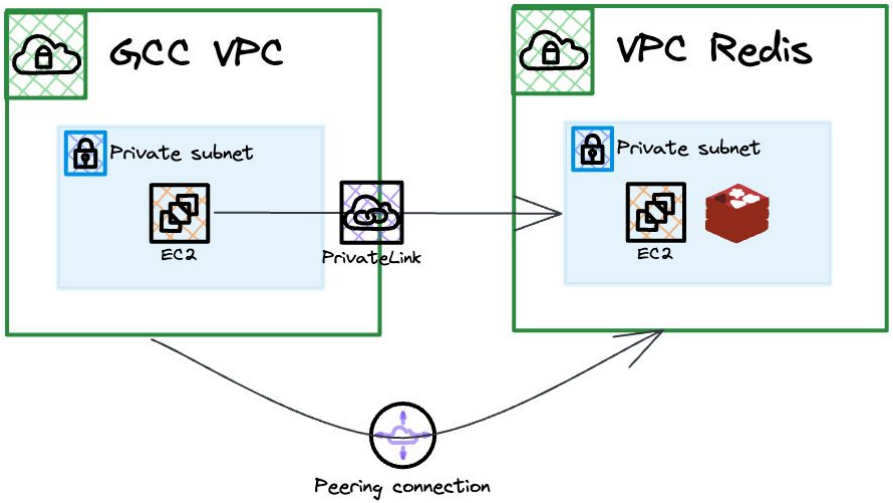
- Runs on customer managed VMs (GCC Cloud or on-prem)
  - Support K8s
- Can operate within secured private subnets
- Integrates with existing IAM and network controls
- No outbound public traffic needed



Deployment Option: Redis Cloud

Fully managed Redis Enterprise offered as a service

- Simplifies operations
- Meets high availability and performance requirements
- Customer controls residency and security boundaries



Deployment Option: Redis Cloud

Private Link / Private Service Connect

- Private, Zero-internet connectivity
- Traffic stays within GovTech-approved private routing
- Do not want to expose or managed routes to another VPC's whole CIDR

VPC Peering

- Direct VPC to VPC Connection - via route table
    a. Potentially expose all VPC network to redis - Broader network exposurer
- Traffic stays within GovTech-approved VPC
- No granular access control
    a. All allowed security group rules apply across the whole VPC space

Deployment Option: Redis Cloud

Data Residency

- Cloud Provider (GCP, AWS)
- Region to meeting Singapore data residency requirements
- Multi-zone redundancy within the chosen region (4 9s SLA)
- All data, backup, metadata remains within selected region

When to Choose Redis Software or Redis Cloud?

Redis Software

- Strict isolation / air-gapped environment
- On-prem or hybrid setup
- Agencies requiring deep customisation or offline operations
- Full control over security posture and operational processes

Redis Cloud

- Minimal operational overhead
- Auto Scaling and built in HA
- Private connectivity and data residency control
- Redis ACL control

Redis Key Features

Redis Query Engine (RQE)

Redis Query Engine Capabilities

Vector Search

INDEX TYPES

Flat (KNN)

HNSW (ANN)

SVS Vamana

QUERY TYPES

Hybrid

Pre-filtering

DISTANCE METRICS

Euclidean

Cosine

Inner product

VECTOR TYPES

FP32

FP16 / BF16

Int8

Full-text Search

SEARCH CAPABILITIES

Stemming

Stop Words

Spell-checking

Fuzzy

Synonyms

Proximity

SCORING ALGORITHMS

BM25

TF-IDF

DISMAX

Hamming

Other Search Types

Numeric

Geo-Spatial

Polygon

Tags

Unlock the power of your data with query, sorting, and analysis

What you're trying to do

- → Power fast business analytics
- → Provide real-time search to your app's users
- → Enable fast location-based queries
- → Quickly and efficiently match users with real-time matchmaking

How this solution can be used

- Deduplication
- Full-text search
- Geospatial search
- Query
- Probabilistic
- Secondary indexing

How we help

Performance
Search, query and analyze your data in real-time.

Scalability
Maintain consistent search performance at any scale scale.

Redis Query Engine
Index, aggregate, search, and query popular data types like JSON and vector.

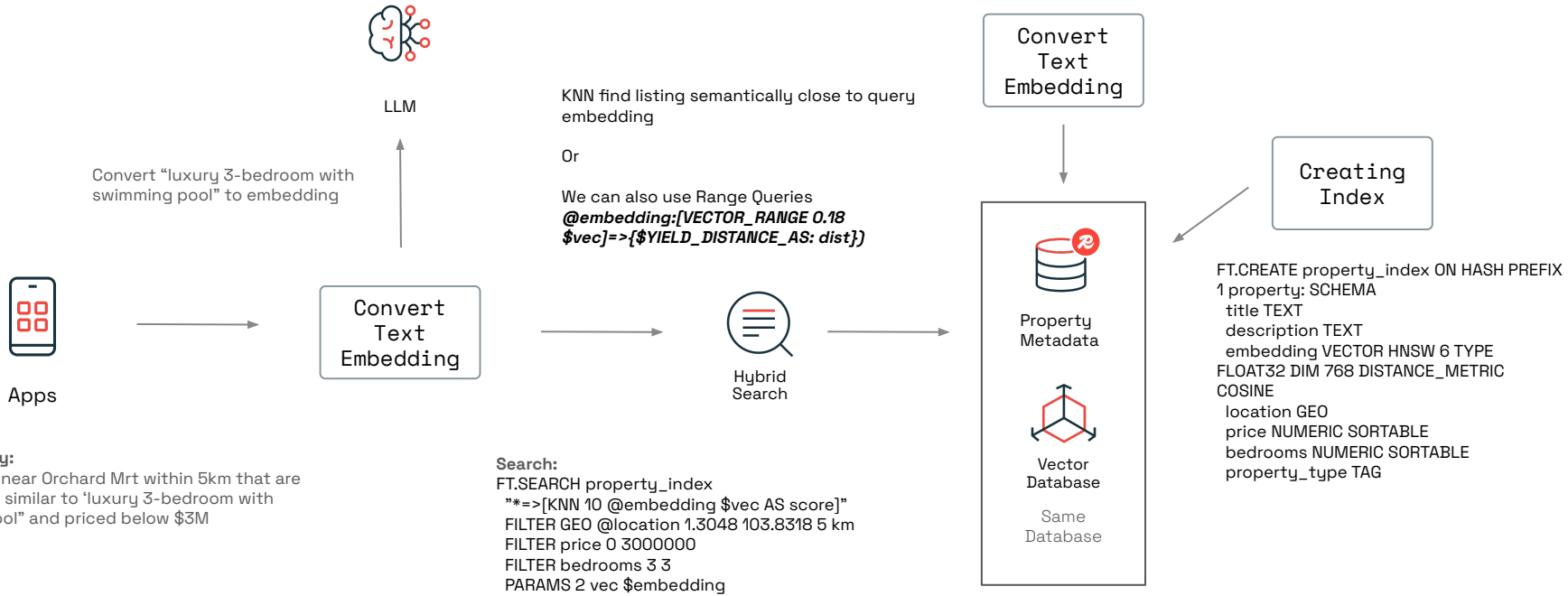
Probabilistic data types
Handle large-scale data with approximate for improved memory efficiency.

Geospatial indexing
Query and filter data based on geographic locations.

RDI
Seamlessly sync data from your database into Redis to enable real-time query, sorting, and analysis.

Hybrid Search: Search + GeoLocation

- Semantic Field - title, description (embed as vectors for semantic search)
- Geo Fields: Lat/Long of the properly (radius, bounding box)
- Structured metadata: Price, number of bedroom, floor area



FT.CREATE property_index ON HASH PREFIX 1 property: SCHEMA
title TEXT
description TEXT
embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 768 DISTANCE_METRIC COSINE
location GEO
price NUMERIC SORTABLE
bedrooms NUMERIC SORTABLE
property_type TAG

Sample Query:
Find condos near Orchard Mrt within 5km that are semantically similar to 'luxury 3-bedroom with swimming pool" and priced below $3M

Search:

```
FT.SEARCH property_index
  "*=>[KNN 10 @embedding $vec AS score]"
  FILTER GEO @location 1.3048 103.8318 5 km
  FILTER price 0 3000000
  FILTER bedrooms 3 3
  PARAMS 2 vec $embedding
  SORTBY score
  DIALECT 2
```

RQE vs OpenSearch (Technical Characteristics)

Technical Aspect	RQE	Open Search
Deployment Model	Embedding search engine running inside redis, no separate nodes or cluster lifecycle to manage	Runs as an independent distributed search cluster with dedicated master, data and ingest nodes
Indexing Architecture	In memory columnar + inverted indexes stored directly in Redis memory space with on disk persistence	Lucene-based inverted indexes stored on disk, indexing involves segment writes, merges and compaction
Latency Characteristics	Memory resident index structures → Consistent sub-milliseconds query path (Ideal for synchronous API calls)	Queries traverse lucene segments → typical 10-50ms depending on I/O and cluster load

RQE vs OpenSearch (Technical Characteristics)

Technical Aspect	RQE	Open Search
Scaling Mechanism	Scaling handed by Redis enterprise auto sharding, search indeed redistribute automatically with slot migration	Requires manual shard, replica and routing strategy design, poor shard planning leads to hot spots
Operational Overhead	Cluster tuned by RE operations limited to DB sizing and throughput config	High requires: JVM tuning, heap sizing, GC configuration, shard sizing, index lifecycle management
Query Capabilities	High-performance, full-text, vector similarity, JSONPath-style filtering. Support hybrid queries and real-time search over redis data models	Rich search ecosystem with deep aggregations. Optimized for broad analytical search workloads
Data Profile	Optimized for hot operational datasets (MB-GB scale) with ultra low latency patterns. Works best when indexes fit entirely in memory	Optimized for warm/cold analytical datasets (100GB-PB scale) where disk storage are acceptable trade-offs for deeper analytics

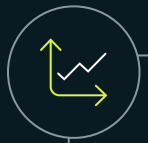
Redis Data Integrator (RDI)

Fast apps are non-negotiable. Whatever your industry, your users are demanding speed.

Responsive retail apps

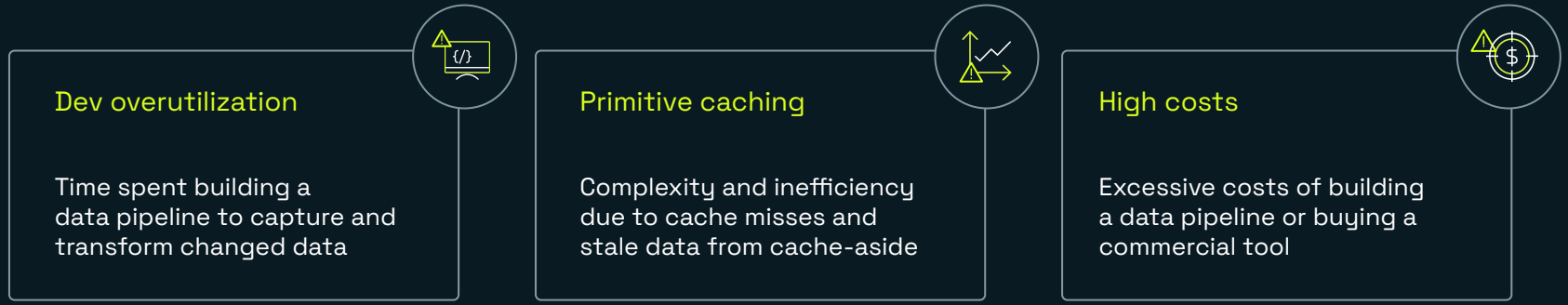
Fast banking apps

Real-time inventory systems



Financial trading platforms

But syncing data between your database and Redis can be complex & expensive.



With so many challenges, fast apps might feel out of reach.

With RDI, these challenges don't hold you back.

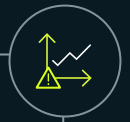
Dev overutilization

Time spent building a data pipeline to capture and transform changed data

✓

Make slow data fast—with ease

Seamlessly build fast, scalable apps pairing Redis' speed with your existing databases



Primitive caching

Complexity and inefficiency due to cache misses and stale data from cache-aside

✓

Cache more than ever before

Go beyond cache aside, prefetch data to extend fast reads to apps that don't write to the DB

High costs

Excessive costs of building a data pipeline or buying a commercial tool

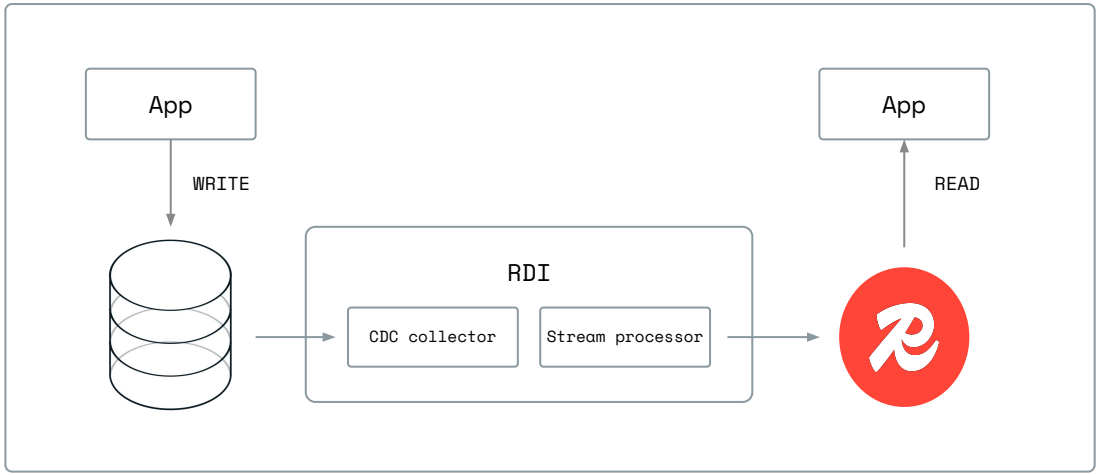
✓

Avoid excessive database costs

Use an out-of-the box data pipeline to offload reads to Redis—and skip expensive database read-replicas

Seamlessly sync Redis with your database.

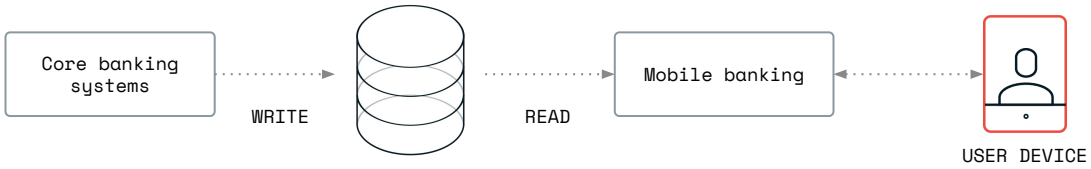
- → Effortlessly configure and deploy a data pipeline between Redis and your other slower databases.
- → Capture, ingest, and transform changed data with configuration —not code.
- → Easily deploy, manage, and visualize your data pipeline in Redis Insight or the Redis Cloud console.



How RDI works

1. **RDI initially populates** Redis with data from your DB
2. **A change data capture (CDC) collector** keeps track of any changed data written to your database
3. **RDI stream processor** translates the data to the preferred data model and types
4. **The data is ingested into Redis** and can be read from your now faster app

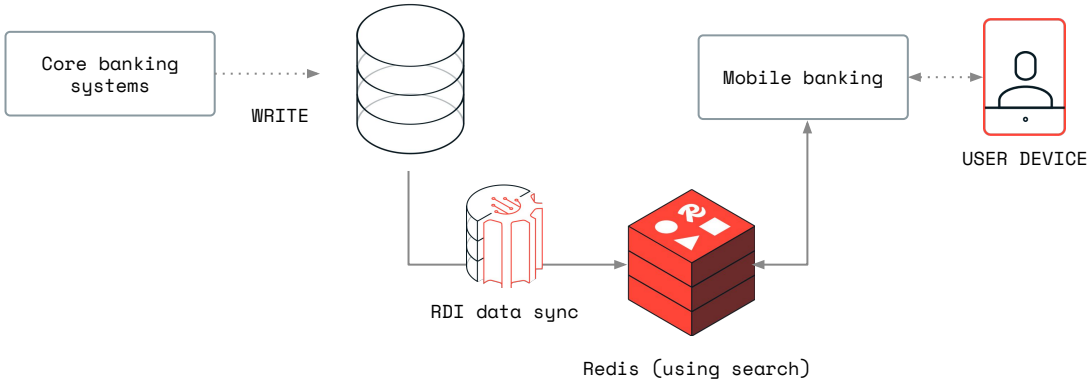
Axis Bank: Before and after



BEFORE

Querying traditional DB

- **App response times:** 170 ms
- **Growth mechanism:** Adding read replicas without performance improvements



AFTER

Syncing data with RDI and querying Redis

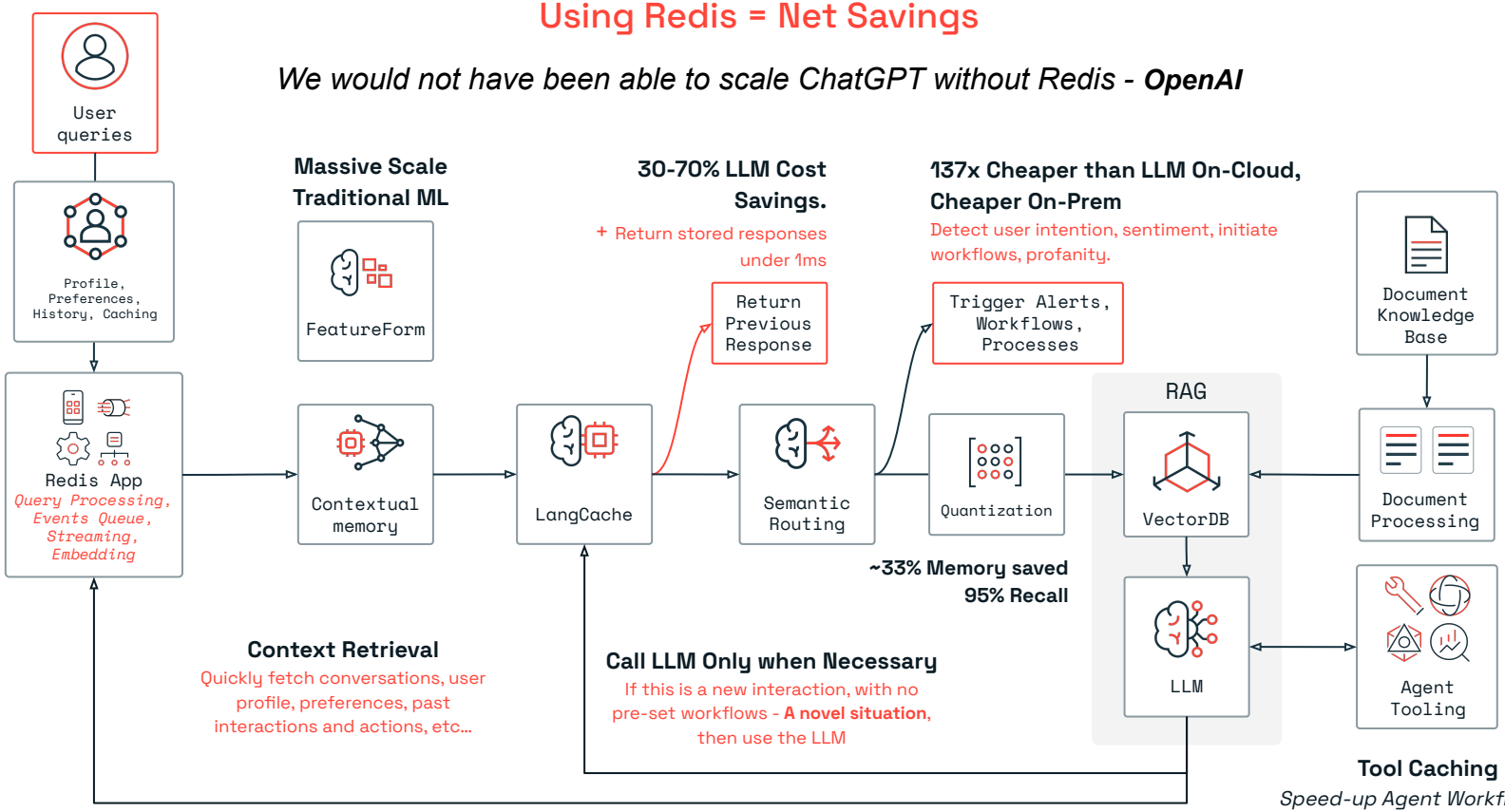
- **App response times:** 40 ms
- **Growth mechanism:** Adding shards with predictable great performance

Redis For AI

The Redis AI Vision

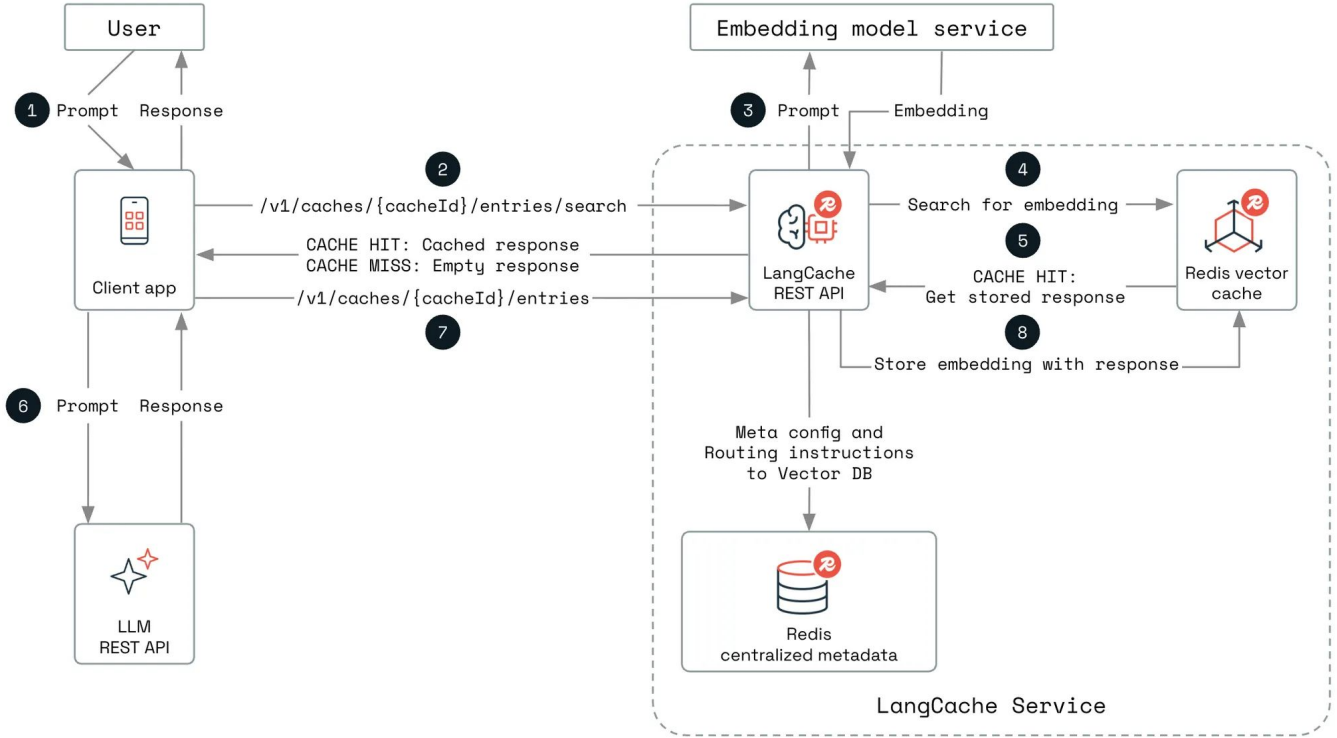
Using Redis = Net Savings

*We would not have been able to scale ChatGPT without Redis - **OpenAI***



LangCache

Stop calling your LLM Model for every request



Deploying GenAI Systems at Scale is Very Hard

Costs add up as usage grows

Avg cost per RAG Call (GPT-5):

0.004286 USD

Based on OpenAI API pricing and average prompt lengths per Gan et al. RAG-MCP (2133 Prompt, 162 Output)

A bank may have 50,000 employees. It builds a knowledge base.

If each employee uses it five times per day:

$1071 per day, $33,021 per month, $398,598 per year

We should try very hard to save 31% ($123,565)

High latency impacts UX and performance

Avg Latencies for LLMs:

GPT-4

0.615s to start, 0.026s per token,
5.2s/200 tokens

Mistral

0.495s to start, 0.041s per token,
8.2s/200 tokens

Claude

1.162s to start, 0.049s per token
9.8s/200 tokens

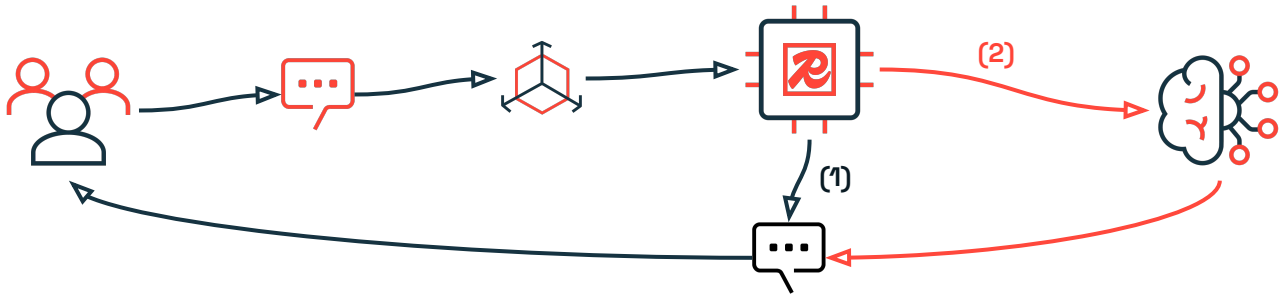
Per AIMultiple Latency Benchmark

Solution: Semantic Caching with Redis

Semantic Caching is just good housekeeping for any LLM system.

(A) Query vectorized, Redis semantic search pipeline applied.
Task: Find similar past responses and rank.

(B) If similar response in cache, return it **(1)**
If no similar response, call LLM **(2)**
Tune similarity threshold to control behavior



Real Life Use-case:

A major bank found **~51.7-71.8%* of queries trigger cached responses** for customer chatbots.

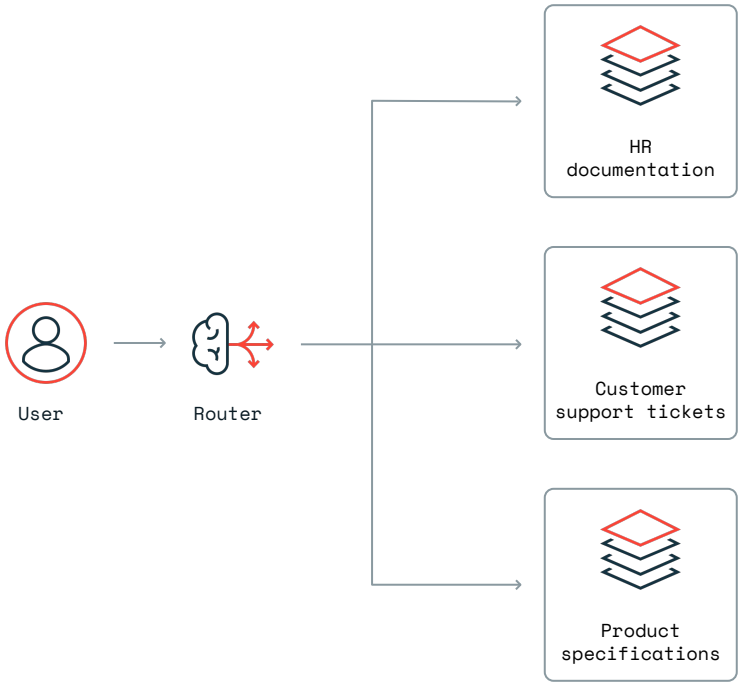
That translates to **similar savings on LLM usage**, while responses came back in **milliseconds instead of ~5 seconds** — up to **1000x faster**.

Semantic routing

Direct AI queries based on meaning, not just keywords. Using vector search, apps understands intent and routes queries to the best data source, tool, model, or processing route.

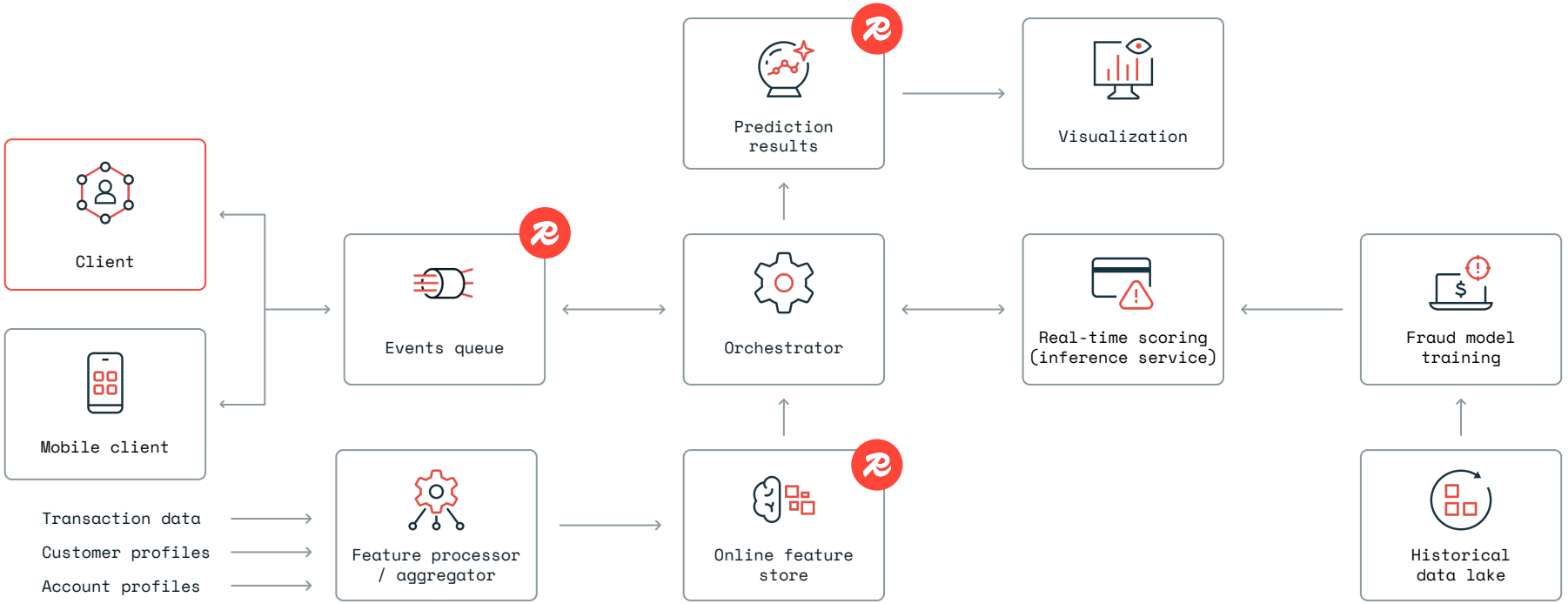
→ **Reduce tokens** by choosing the optimal LLM

→ **Protect your company** by adding guardrails for bad behavior



Anomaly & fraud detection

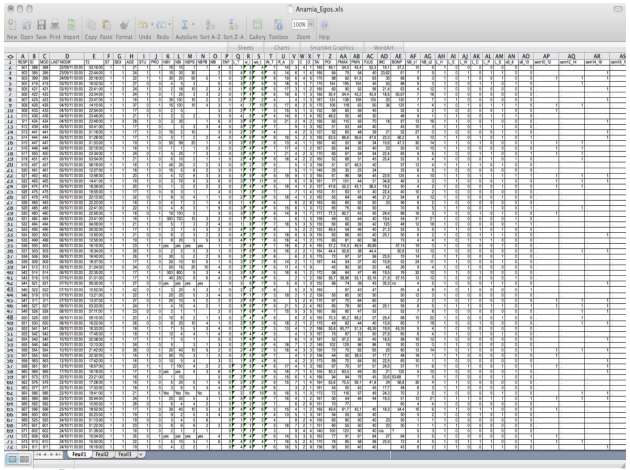
Stop the bad guys in their tracks.

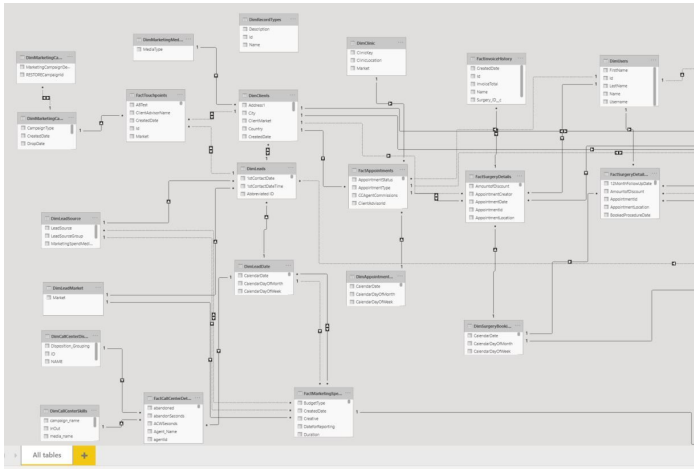


An Extreme ML Domain

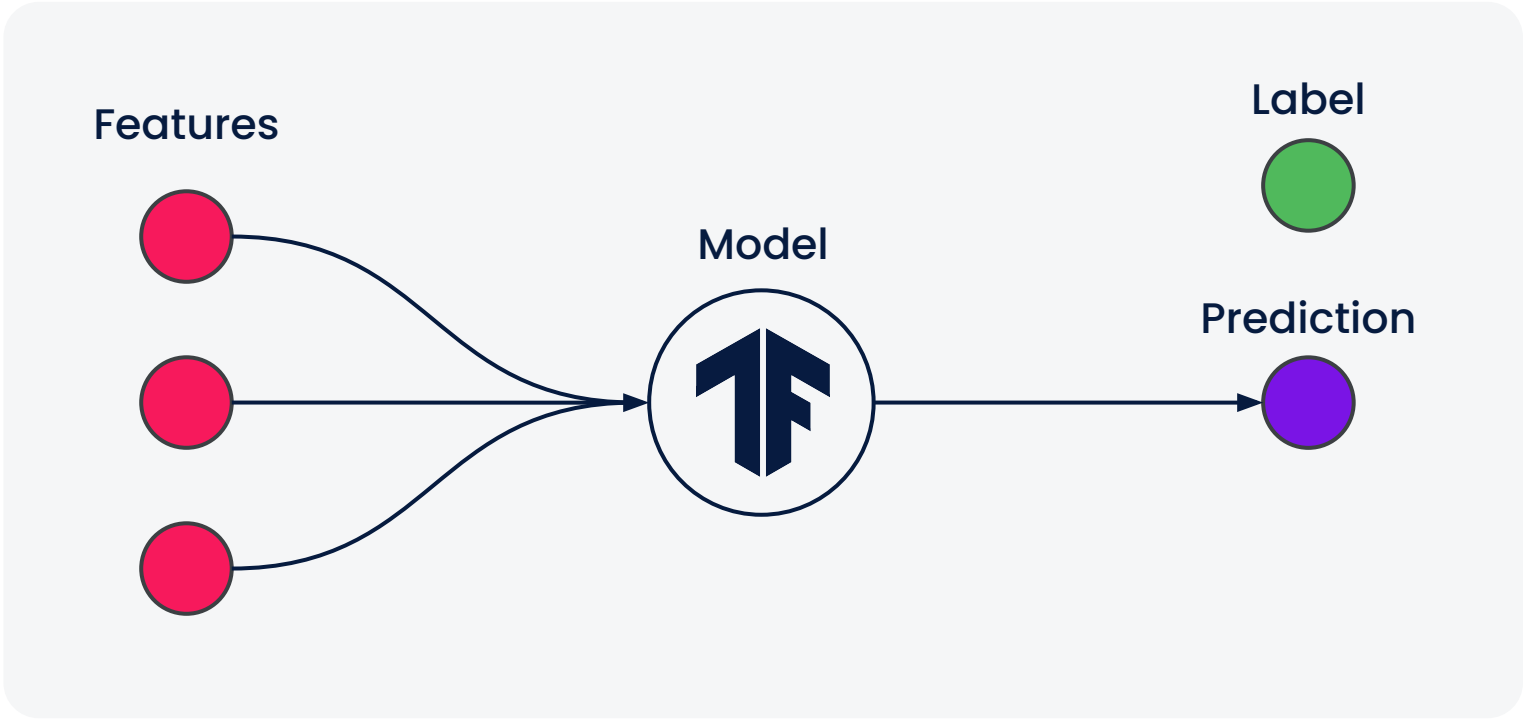
Fraud Detection requires a Formula 1 engine, not a chatbot.

- **Speed:** Each transaction must be judged in milliseconds.
- **Scale:** Millions per second.
- **Reliability:** You **cannot** have hallucination! An LLM has **NO** guardrails. It could cause the next BoA Failure!
- **LLMs:** Built for human conversation. This is not a world for humans.





Feature are inputs to models



Solution - Step 1: Feature Transformation

Feature Transformation -> From base data, we extract the patterns relevant for fraud detection

Base Data

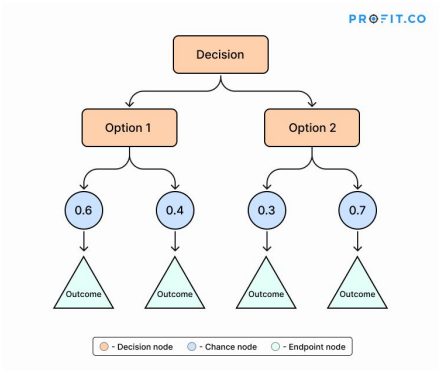
Field	Example	Notes
txn_id	T123456	Unique transaction
user_id	U001	Customer identifier
card_id	C789	Payment instrument
device_id	D456	Device fingerprint
ip_address	10.1.2.3	Source IP
amount	85.40	Transaction amount
currency	USD	Standardized
merchant_id	M321	Merchant reference
timestamp	2025-10-23 12:45:32	Event time

Transformed Data

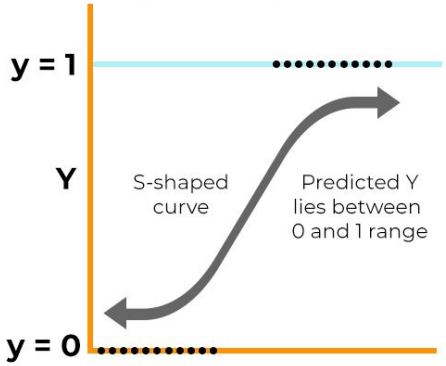
Entity	Window	Derived Metric	Example Output	Transformation
user_id	5 min	txn_count_5m	3	Count transactions per user in 5 min
user_id	24 h	amount_sum_24h	860.50	Sum amounts per user in last 24 h
device_id	7 d	distinct_users_7d	5	Count unique users per device
ip_address	1 h	geo_variance	250 km	Haversine distance between IP locations
merchant_id	30 d	chargeback_rate	0.012	(#chargebacks / #sales)
card_id	24 h	declines_24h	2	Count declined transactions per card

Solution - Step 2: Create ML Models. Each will capture a specific patterns

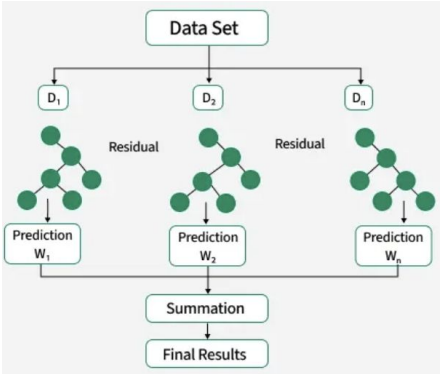
Decision Trees



Regressors



XGBoost

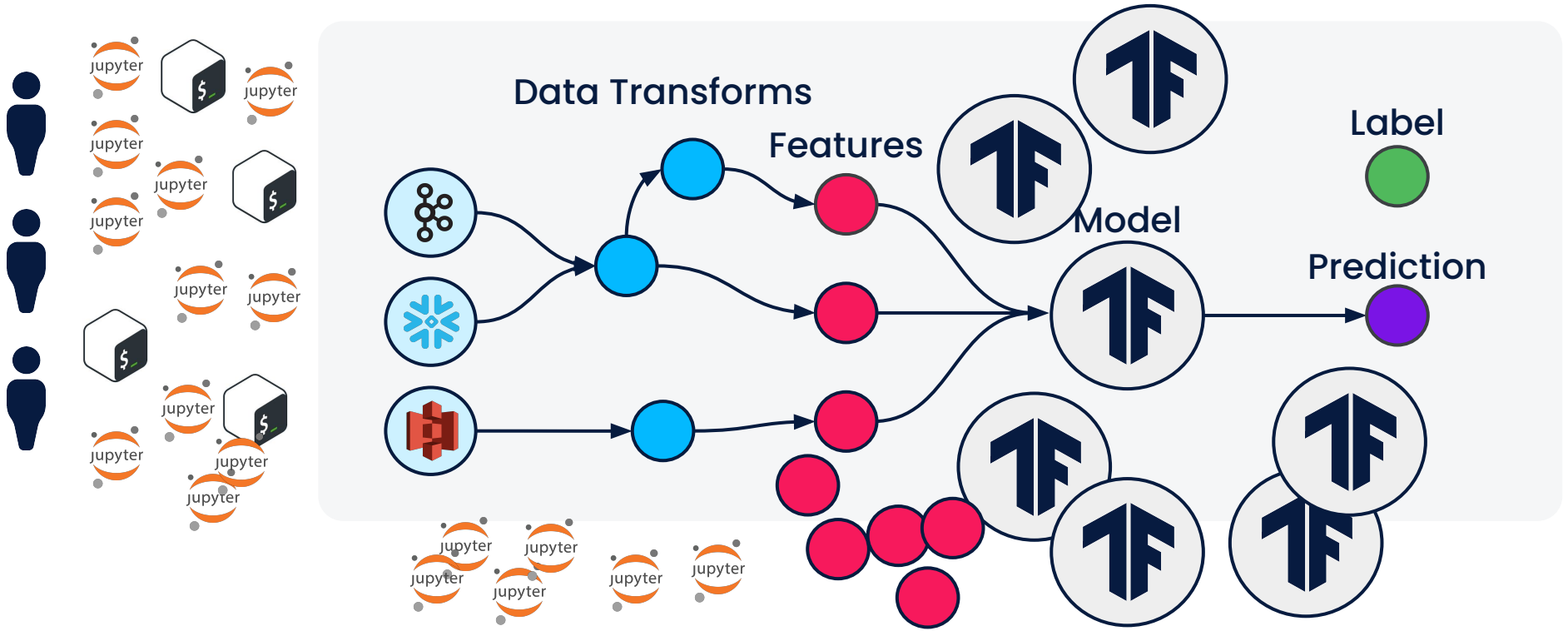


Ultra Fast, Ultra Light, One Specialized Role

Thousands of classifiers deployed.

Tens of Millions of Predictions per second - This is the scale BCA needs.

Problem



Thankfully, we have the solution.

Redis welcomes FeatureForm to the fold

How Featureform hit $1.3M revenue with a 12 person team in 2025.

Featureform turns features into a first-class component of the machine learning process.

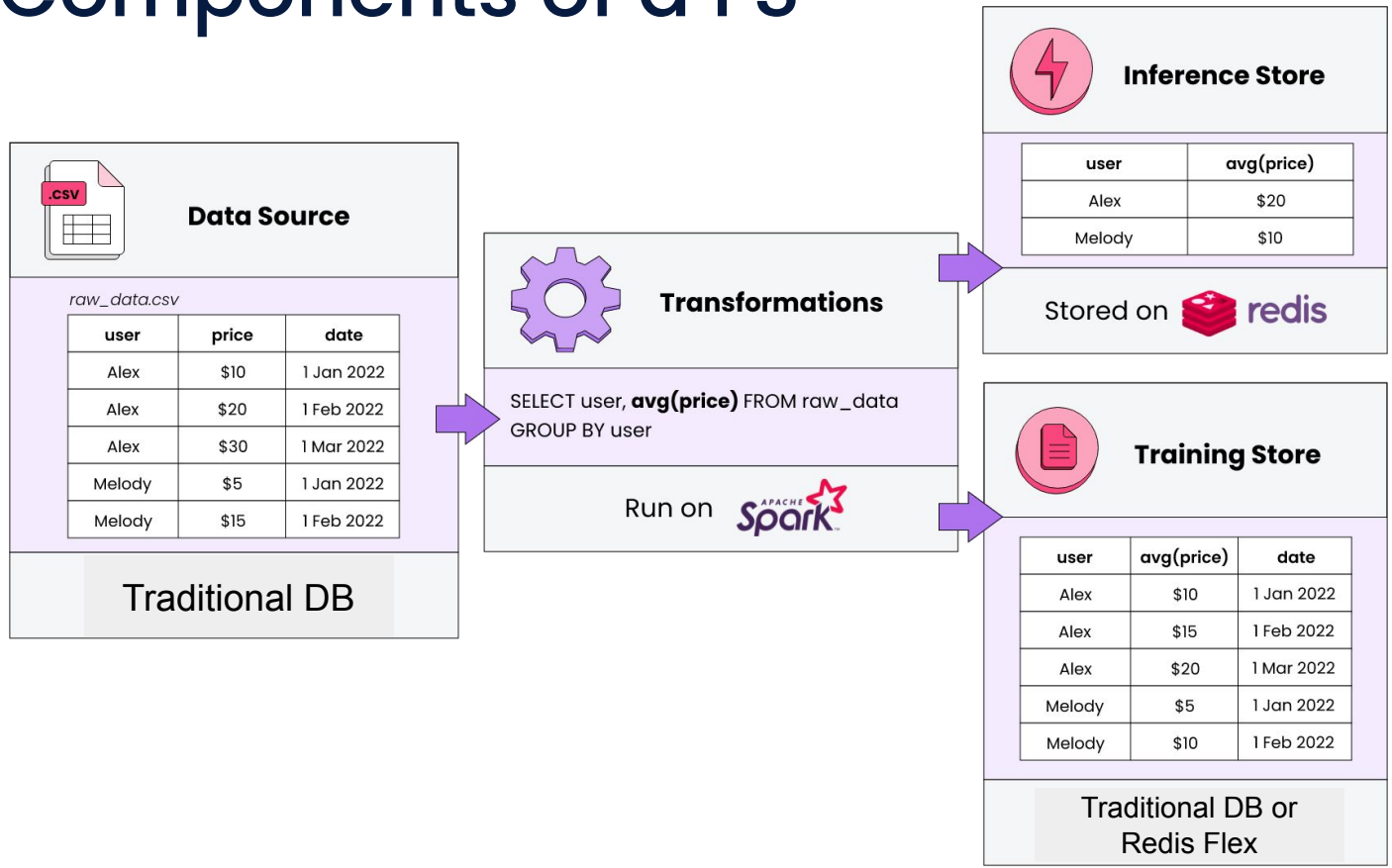
Featureform is a featurestore.

- Ultra-fast data transformation
- Live data serving to production ML models.
- Highly reliable and resilient. Redis 99.999% uptime guarantee.
- Standardized, repeatable workflows, with versioning.

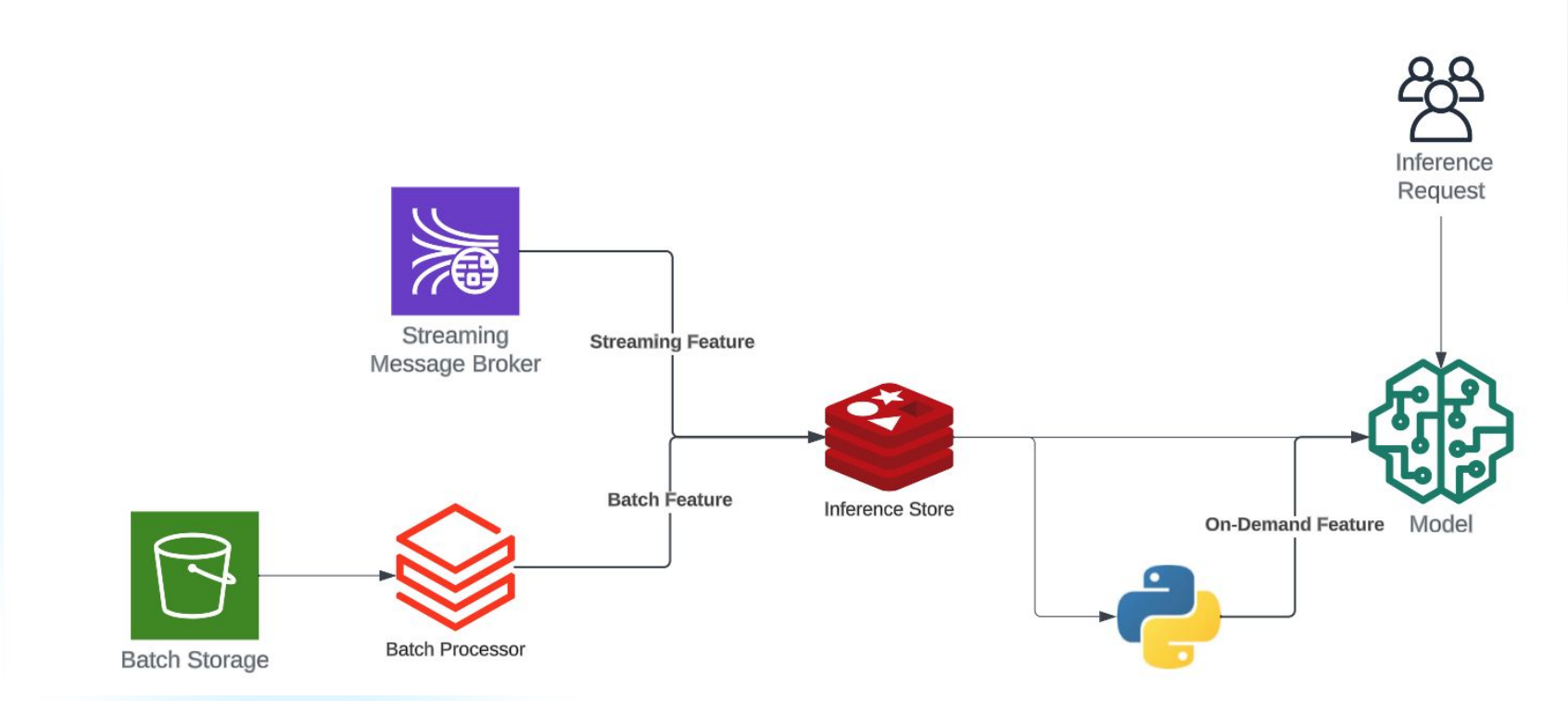
The Scale BCA Needs:

- Millions of transactions per second
- Thousands of Models
- Dozens or hundreds of team-members
- How will you manage all of this?

The Components of a FS



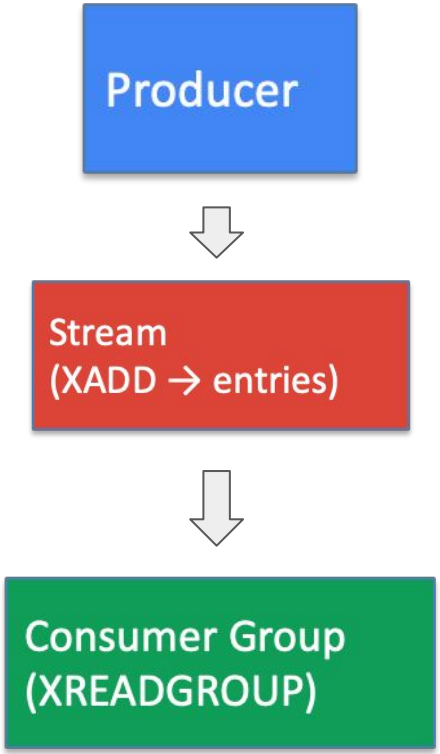
Production Features



Redis Stream

Redis Stream Overview

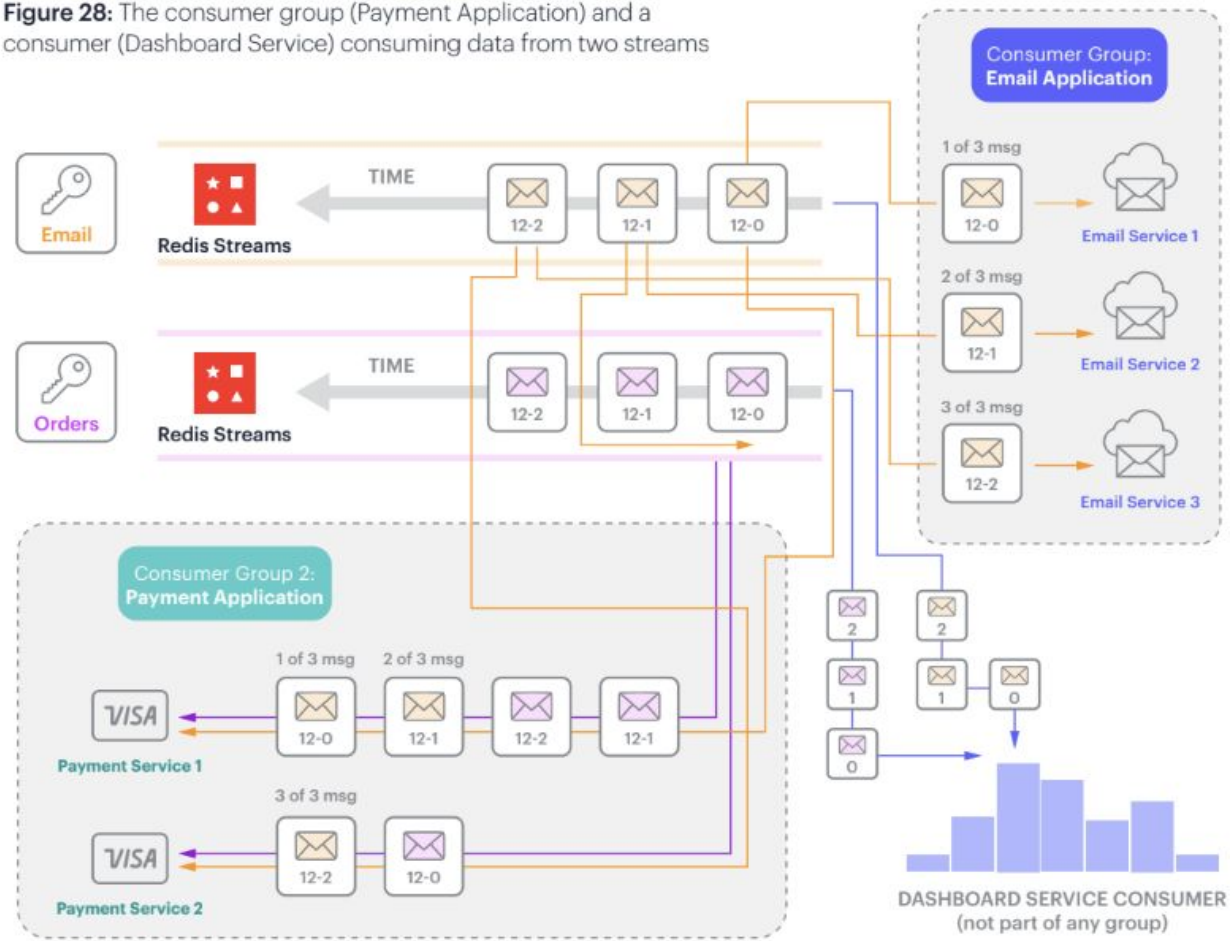
- Log-based append-only, log-based data structure designed for event streaming, message queuing and real-time data pipelines
- A persistent, ordered log of events that multiple consumers can read at their own pace”
- Each entry has:
    - Unique ID (timestamp based)
    - A set of key-value fields



Redis Stream Benefits

- **High-throughput event digestion**
  - Millions of operations per second
  - Low Latency writes
- **Built-in fan-out with Consumer Groups**
  - Multiple consumers can read the same stream, but each message can be delivered once access the group
  - Horizontal Scaling, Failover (Unacknowledged messages can be reclaimed)
- **Persistence + History retention**
  - Unlinke Pub/Sub, Streams store history
  - Clients can - replay events read ranges of data, process from any offset
- **Back-pressure control**
  - Consumers read at their own pace
    - Slow consumers won't block fast ones
    - Redis won't drop messages unless configured to

Figure 28: The consumer group (Payment Application) and a consumer (Dashboard Service) consuming data from two streams



thank you